

# Autodata: 一种用于创建高质量合成数据的智能体数据科学家

Ilia Kulikov<sup>†</sup>, Chenxi Whitehouse<sup>†</sup>, Tianhao Wu<sup>†</sup>, Yixin Nie<sup>†</sup>, Swarnadeep Saha, Eryk Helenowski, Weizhe Yuan, Olga Golovneva, Jack Lanchantin, Yoram Bachrach, Jakob Foerster, Xian Li, Han Fang, Sainbayar Sukhbaatar, Jason Weston

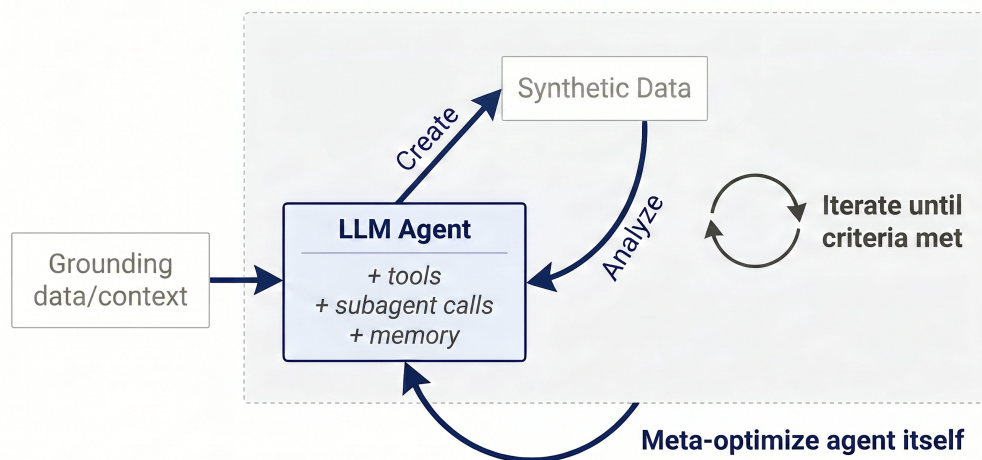
FAIR at Meta <sup>†</sup>Joint first author

本文提出 Autodata，一种通用方法，使人工智能智能体能够充当数据科学家，构建高质量的训练与评估数据。本文展示了如何训练（元优化）此类数据科学家智能体，使其学会创建更强大的数据。本文描述了整体框架，以及一种具体的实用实现——智能体自指导（Agentic Self-Instruct）。本文在计算机科学研究任务、法律推理任务以及数学对象推理上进行了实验，与经典合成数据集创建方法相比，取得了更优的结果。此外，对数据科学家智能体本身进行元优化带来了更大的性能提升。智能体数据创建提供了一种将增加的推理计算转化为更高质量模型训练的途径。总体而言，本文认为这一方向有潜力改变我们构建人工智能数据的方式。

Date: July 7, 2026

Correspondence: [kulikov@meta.com](mailto:kulikov@meta.com), [jase@meta.com](mailto:jase@meta.com)

Meta



**图 1 Autodata 流程。**该框架采用一个自主智能体，模拟数据科学家的角色，迭代地生成数据、进行定性检查和定量性能评估、综合洞见，并更新数据生成配方。该智能体本身可以使用与内循环相同的标准进行训练，以更好地胜任数据科学家角色。这一循环过程旨在逐步提升数据质量；该图描绘了可能实例化所依据的通用工作流程。

## 1 引言

人工智能前沿的进展越来越依赖于高质量的训练数据和持续挑战模型的基准。当前人工智能系统训练的初始基础是人类编写的训练数据。然而，性能提升越来越多地来自模型自身创建的合成数据。合成数据解决了若干实际挑战：它有助于生成真实语料库中代表性不足的边缘情况和长尾场景，降低了人工标注的难度和延迟，

并且有可能产生比人类生成的数据分布更具挑战性的数据。

随着大语言模型（LLMs）的引入以及上下文学习和指令遵循能力的实现，自指导（Self-Instruct）（Wang 等，2023）作为一种通过零样本或少样本提示创建合成数据的方法应运而生。基于文档的自指导（Grounded Self-Instruct）（Lupidi 等，2024；Yuan 等，2025）将其扩展为基于文档和其他来源，以减少幻觉并增加多样性。此外，思维链自指导（CoT Self-Instruct）（Yu 等，2025）等方法将其扩展为在生成过程中使用思维链推理，以帮助更准确地构建更复杂的任务。最后，所谓的“自我挑战”方法（Zhou 等，2025）允许挑战者智能体在提出任务和配套评估函数之前与工具交互。然而，这些方法都没有直接控制数据的难度和质量，这促使了过滤（Yu 等，2025）、进化（Xu 等，2024）和精炼（Shah 等，2024）等方法的出现。

在本文中，我们提出了自动数据（Autodata），它概括了上述所有方法。一个充当数据科学家的智能体负责构建和管理数据，执行人类数据科学家为创建高质量数据而采取的行动：其中构建基准数据和训练数据都是用例。这个过程包括初始的数据创建迭代，随后是分析阶段，即“目测”数据并测量其性能，构建经验教训，然后用改进的方案进行迭代以创建更好的数据。此外，我们展示了如何训练（元优化）这个智能体系统（外循环），使其作为数据科学家（内循环）达到最优。虽然最近关于自动研究（Karpathy，2026）的许多工作都集中在用于架构或训练方案改进的智能体方法上，但我们认为，专注于数据很可能在未来进展中发挥同等重要甚至更重要的作用。

在我们的实验中，我们专注于自动数据的一个特定实现，即智能体自指导（Agentic Self-Instruct），并表明它在各种任务中提供了强大的结果。我们在计算机科学研究任务、法律推理任务和数学对象推理上进行了实验，在每种情况下，与经典的合成数据集创建方法相比，我们都获得了改进的结果。此外，对数据科学家智能体本身进行元优化带来了更大的性能提升。

随着最先进的大语言模型（LLM）日益强大，人们担忧现有任务或合成数据方法无法生成足够具有挑战性的任务以推动进一步进展。自动数据（Autodata）通过智能体式数据创建，提供了一种将增加的推理计算转化为更高质量模型训练的途径，从而生成此类具有挑战性的数据。因此，本文认为该方向有潜力改变我们创建新任务和基准以推进前沿的方式。

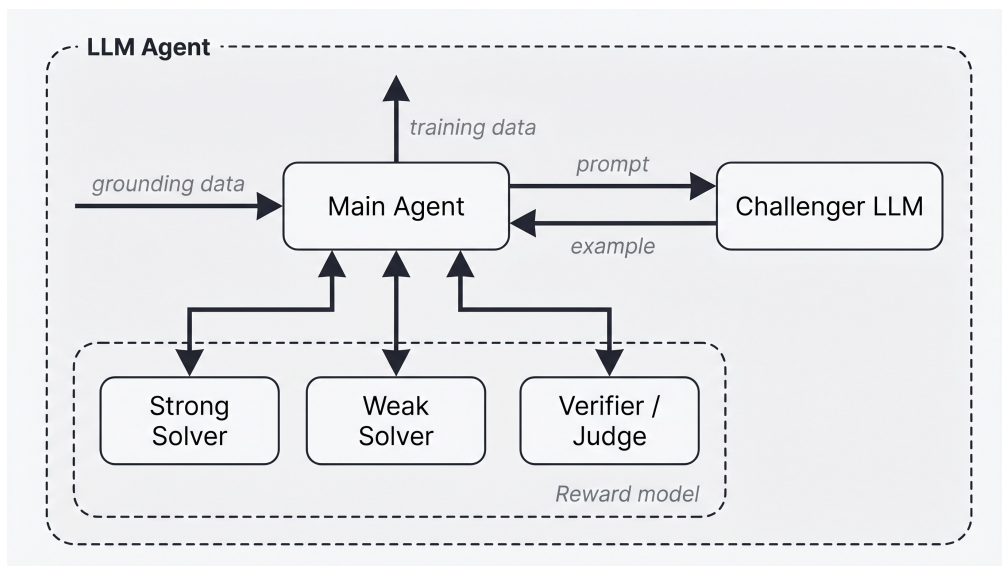
## 2 自动数据

自动数据的高层设计如图 1 所示，其中可以从该模板构建各种实例化。整体循环由以下组件组成。

**数据创建。**自动数据智能体基于一些提供的数据（例如，来自数学、法律、编码等领域的特定文档，或根据任务选择的其他有用数据源）来帮助创建数据。然后，该智能体可以使用工具或先前获得的现有技能/知识以及推理时间计算，为模型训练和基准测试创建训练或评估数据。重要的是，此创建步骤可以在后续分析和学习之后重复进行，以进一步改进数据。

**数据分析。**给定智能体已创建的数据，它可以分析这些数据，以了解哪些做对了、哪些做错了，以及如何改进。这可以在具体示例层面进行（检查示例是否正确？质量是否高？是否足够具有挑战性？），也可能在数据集层面进行（样本是否多样？如果用作训练数据，它们是否能改进模型？）。这些学习结果被反馈到数据创建过程中，以在下一轮迭代中改进数据，直到满足停止条件。

**整体数据科学家循环。**智能体在数据创建和数据分析阶段之间循环，直到对数据质量满意为止，然后生成最终的训练数据集或基准。外层循环中可以加入特定的护栏机制以防止作弊。这种智能体循环使模型能够在这些步骤中基于自身的学习成果进行构建。



**图 2 弱对强智能体自指导方法。**主大语言模型智能体指挥四个子智能体：挑战者大语言模型生成示例；弱求解器和强求解器尝试解答；评判者评估它们的输出。系统旨在生成强求解器成功而弱求解器困难重重的训练数据。主大语言模型分析数据，并利用评判者的反馈更新挑战者提示，重复循环，从而产生用于训练弱求解器的具有挑战性的示例。

**数据科学家的元优化。**智能体本身也可以被优化，以更好地胜任数据科学家的工作。一种方法是利用自动研究（Karpathy, 2026）或元框架（Lee et al., 2026）风格的优化来优化智能体框架，使用相同的内层循环标准（创建更好的数据）来指导外层循环（智能体优化本身）的优化。这在外层方框中有所描绘

图 1。

**2.1 具体实现：智能体自指导** 在我们的实验中，我们考虑了一种具体且实用的自动数据实现，用于创建高质量数据，我们称之为智能体自指导，如图 2 所示。主协调智能体可以访问四个大语言模型子智能体：

- (i) 挑战者，根据主智能体给出的详细提示创建训练示例；
- (ii) “弱”求解器，预期通常在解决所创建的训练数据时会遇到困难；
- (iii) “强”求解器，预期通常能成功完成所创建的训练数据，
- (iv) 验证器/评判器，给定示例和模型解答，检查其质量，并将其学习结果反馈给主智能体。

主智能体随后着手创建一个示例（例如，根据任务给定的上下文/输入、期望响应或参考答案以及评估标准），通过向挑战者发送包含基础上下文的初始提示。然后，它通过将输入发送给弱求解器和强求解器来检查挑战者输出的质量，并根据验证器的判断分配奖励。评判器还负责检查示例本身的质量：问题、参考答案或生成的评分标准。

对于可验证任务（使用基于大语言模型的验证器），一种方法是要求强求解器上的多数投票正确，而弱求解器上的多数投票错误。对于不可验证任务，我们要求由评判器衡量的质量差距，例如挑战者生成的评分标准，使得任务对弱求解器既不太容易也不太困难，同时强求解器有助于保证正确性。智能体分析验证器（包括求解器输出）的报告，如果未满足此标准，

则它继续根据这些新学习结果修改发送给挑战者的输入提示，尝试创建新示例，直到满足标准。

这一过程使智能体能够有效地学习如何专门为训练“弱”求解器创建具有挑战性和高质量的示例。我们注意到，“弱”和“强”求解器实际上可以是同一个大语言模型，但处于不同模式，例如，强版本可以允许使用增加的推理时间计算，包括脚手架或聚合（Zhao 等人，2025b），以及访问特权信息。

## 3 实验

### 3.1 计算机科学研究任务

我们考虑回答计算机科学（Computer Science, CS）研究问题的任务，以学术计算机科学论文作为源材料。计算机科学研究问题是开放式的，与可验证任务不同，需要基于评分标准的评估。挑战者生成上下文、问题、参考答案以及一个自包含的评估评分标准，该标准由加权准则组成，大语言模型裁判在不访问参考答案的情况下使用该标准对任何回答进行评分。

**表 1 生成的计算机科学研究任务的质量统计。**思维链自指导（CoT Self-Instruct）是标准提示生成；智能体自指导（Agentic Self-Instruct）是智能体循环接受的输出。两列均在生成时由 Kimi-K2.6 在相同的 4B 弱求解器/397B 强求解器对上评分。

| Metric                  | CoT Self-Instruct | Agentic Self-Instruct |
|-------------------------|-------------------|-----------------------|
| Weak solver avg         | 0.677             | 0.458                 |
| Strong solver avg       | 0.696             | 0.772                 |
| Gap (strong – weak)     | 0.019             | 0.314                 |
| Agentic rounds          | 1.00              | 6.59                  |
| Question length (chars) | 723               | 619                   |
| Rubric items            | 13.2              | 13.1                  |

**流程概述。**主协调智能体调用挑战者，从给定论文生成上下文-问答对及相应的评分标准。质量验证器随后检查上下文泄漏、评分标准覆盖范围和问题质量，再进行最终评估（并作为循环结束时的最后一步）。问题和上下文被发送给两个求解器（每个调用 3 次以减少方差），裁判根据评分标准逐项对它们的答案进行评分。如果任何接受准则失败，智能体向挑战者提供针对性反馈：哪些先前问题太容易（附弱求解器得分），哪些在强求解器上失败（附差距信息），以及哪些被质量验证器拒绝。挑战者随后从不同的推理角度生成新问题。该循环通常每篇论文运行数轮，直到找到被接受的问题或耗尽步骤预算。我们使用 Kimi-K2.6 作为主协调智能体和挑战者，Qwen3.5-397B-A17B 作为强求解器，Qwen3.5-4B 作为弱求解器。智能体系统提示见附录 C.1 小节。

**准则。**对弱求解器有用的训练示例要求强求解器在评分标准上的得分显著高于弱求解器。然而，在初步实验中，由提示的 Kimi-K2.6 生成的大多数问题对弱求解器来说过于简单，改进空间有限：如表 1 中“思维链自指导”列所示，这些问题的弱求解器平均分高于 0.67，弱/强差距仅为 0.02。因此，我们直接根据这一差距定义智能体循环的接受标准：仅当强求解器平均  $\geq 0.65$ 、弱求解器  $< 0.5$ 、且强 – 弱差距  $\geq 20$  个百分点时，候选问题才被接受。鉴于弱求解器的强大性能，我们在每次迭代中节省计算量，仅当弱求解器通过其相应的成功标准时，才让评判器评估强求解器。

**数据设置。**我们从 S2ORC 语料库（2022+）（Lo 等，2020）中处理超过 10k 篇计算机科学论文，利用智能体自指导生成 2.8k 个被接受的示例。循环末尾使用 Kimi-K2.6 的质量验证器（该验证器

表 2 计算机科学研究任务上的强化学习训练结果。自动数据智能体自指导方法优于使用标准思维链自指导创建数据。我们使用 GRPO 在来自每个数据源的 1.3k 个示例上训练 Qwen3.5-4B，并在 200 个提示的留出测试集上进行评估。评分基于评分标准，由 Kimi-K2.6 评定。如图 3 所示，智能体数据在整个训练过程中优于思维链数据；我们在表中以第 200 步为例进行说明。

| Response model                                     | CoT test     |              | Agentic test |              |
|----------------------------------------------------|--------------|--------------|--------------|--------------|
|                                                    | mean@3       | best@3       | mean@3       | best@3       |
| Qwen3.5-4B (no additional RL)                      | 0.630        | 0.758        | 0.366        | 0.484        |
| Qwen3.5-4B RL on CoT Self-Instruct data            | 0.727        | 0.853        | 0.500        | 0.631        |
| <b>Qwen3.5-4B RL on Agentic Self-Instruct data</b> | <b>0.774</b> | <b>0.894</b> | <b>0.632</b> | <b>0.768</b> |

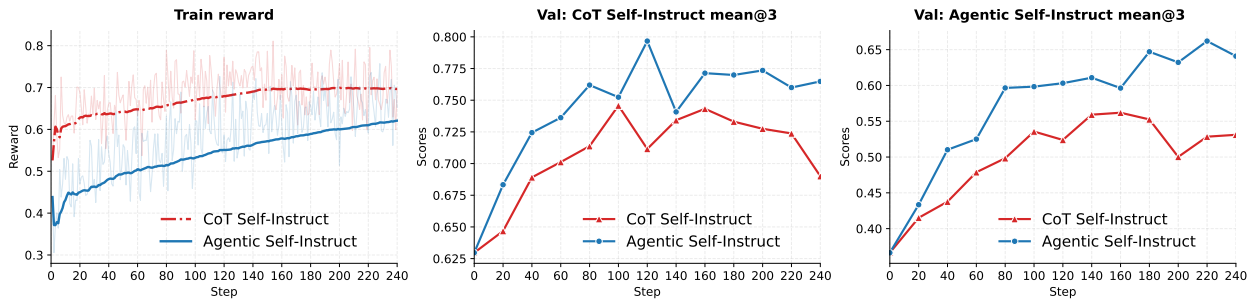


图 3 两种计算机科学强化学习运行的训练奖励和留出验证（Qwen3.5-4B 分别在智能体与思维链自指导数据上训练）。左：每步 Kimi-K2.6 评分标准奖励，经指数移动平均平滑。中/右：思维链和智能体留出测试集上的 mean@3（每个测试集对一个分支是分布内的，对另一个是分布外的）。智能体训练在整个训练过程中在两个留出集上均领先，在更难智能体测试上优势更大。

移除具有论文特定参考泄漏、上下文过短或评分标准格式错误的问题）进一步过滤该集合，我们保留 1.3k 个高质量被接受的示例作为强化学习训练的智能体自指导数据。对于思维链自指导基线，我们也应用相同的质量验证器，并采样等量的 1.3k 过滤数据以进行公平比较。

### 3.1.1 智能体自指导循环分析

在包含一万篇论文的语料库上运行数据创建循环时，智能体需要大量迭代才能生成一个被接受的问题：每个被接受项平均需要 6.59 轮（表 1），且部分论文的迭代轮数超过 10 轮，呈现长尾分布。我们在每一轮观察到的失败模式高度偏向一侧：在 880 轮预接受轮次中，80% 的失败轮次被拒绝是因为问题过于简单，弱求解器得分过高；13% 是因为强求解器也无法可靠地解答这些问题。

对于最终收敛的论文，生成的问题很少是智能体在首次尝试中创建的。例如，弱求解器在基线思维链自指导方法生成的问题上平均得分为 0.677，而在使用智能体自指导方法生成的问题上，基于相同的源材料（论文），得分下降了 22 个百分点。通过检查轨迹，我们观察到智能体对计算机科学论文的初始尝试通常是一个高层次的总结性问题，这类问题对于 4B 求解器模型而言往往过于简单。然而，在后续轮次中，在评判器反馈的引导下，问题逐渐转向论文中的具体算法步骤、消融细节或数值声明，要求遵循论文的实际论证过程。这种搜索的总体效果在表 1 的语料库统计中可见：弱求解器的得分下降了 22 个百分点（0.677  $\rightarrow$  0.458），而强求解器的得分提高了 8 个百分点（0.696  $\rightarrow$  0.772），这证实了被接受的问题在难度上有所提升，且这种难度提升特别有利于强求解器更深层次的推理能力。也就是说，智能体数据创建循环生成的问题专门奖励更强的模型能力，而不是两个模型都能回答的问题。

改进通过探索实现。每一轮智能体都会从不同的推理角度生成一个新问题，并根据反馈了解哪些先前的问题过于简单或未能有效区分模型能力。

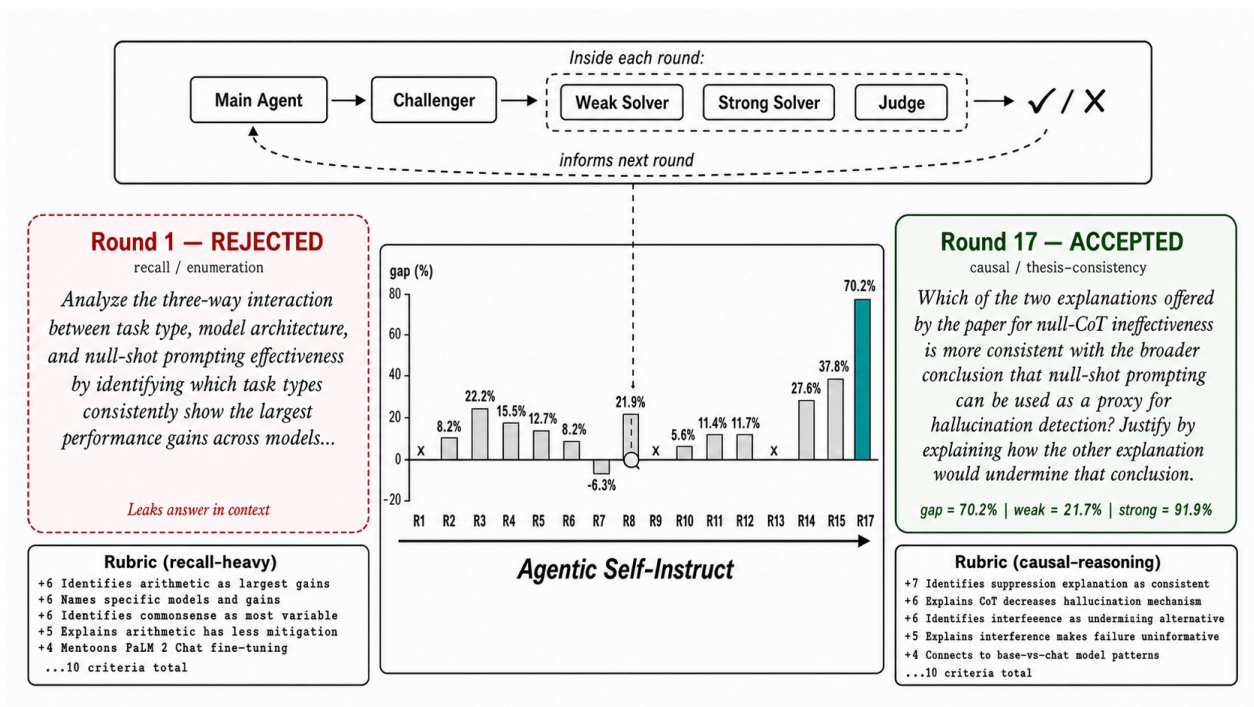


图 4 计算机科学研究问题的自动数据创建。图中展示了智能体自指导方法针对一篇关于大型语言建模的论文生成训练示例及相应评估准则的过程。

经过智能体循环后被接受的问题在定性上也测试了不同类型的推理：具体的技术机制、多步推导以及论文特定的设计权衡，相比之下，未经过该循环生成的问题则更为宽泛和通用。示例见图 4。

### 3.1.2 强化学习训练结果

我们比较了 Qwen3.5-4B 在 CoT 自指导与智能体自指导数据上各训练 1.3k 示例的性能，使用 Kimi-K2.6 作为奖励模型，根据生成的评分标准对回答进行评分。从每个数据集（分别由 CoT 自指导和智能体自指导生成）中，我们使用 100 个示例作为测试集，并用 GRPO (Shao 等, 2024)（批大小 16，学习率  $1e-6$ ）训练 Qwen3.5-4B，在每个留出测试集上评估训练后的模型。

在较简单的 CoT 自指导测试集上（表 2，左），使用 CoT 数据训练将基础 4B 模型从 0.630 mean@3 提升至 0.727，而使用智能体数据训练进一步提升至 0.774。在较难的智能体测试集上（右），对应数值为 0.366（基础）→ 0.500（CoT 训练）→ 0.632（智能体训练）：两种方法之间的差距在此处比在 CoT 测试上大两倍以上，且 best@3 遵循相同排序。图 3 中的训练动态与此一致：我们的智能体方法从开始就在每步 Kimi 奖励上高于 CoT 自指导方法，且差距在训练过程中扩大，而在验证面板上，我们的智能体方法在两个测试集的每个检查点上都匹配或超过 CoT 方法，包括 CoT 数据是自然分布内选择的那个检查点。智能体训练的模型在两个方向上都能迁移（+0.05 到较简单的 CoT 测试，+0.13 到较难的智能体测试），明显的优势表明智能体流程产生的判别性训练数据转化为更强的推理性能。

## 3.2 法律推理任务

我们接下来研究第二个场景，以测试该方法在法律推理任务上的通用性。在 3.1 小节中，我们在开放式计算机科学研究任务上实验了智能体自指导，其中初始合成数据对于下游强化学习训练来说不够具有挑战性，模型无法通过使用这些数据获得改进。本节

研究第二个场景，即提升大语言模型的法律推理能力，结果发现其具有不同的特性。在这里，我们发现智能体自指导必须应对相反的失败模式：我们发现通过 CoT 自指导的标准提示生成的问题太难（而不是太容易），从而阻碍了强化学习奖励信号。

与之前的目标一致，即生成高质量数据以提升弱求解器在给定任务上的表现，此处任务为法律推理。我们使用来自法律语料库（Pile of Law, Henderson 等, 2022）的法院意见书及其他公开法律文件作为源材料，并在 PRBench-Legal 及其困难子集 PRBench-Legal-Hard (Akyürek 等, 2025) 上进行评估。与计算机科学场景相同，我们使用 Kimi-K2.6 作为主要的编排智能体、挑战者和裁判，Qwen3.5-397B-A17B 作为强求解器，Qwen3.5-4B 作为弱求解器。与计算机科学论文截然不同，我们的初步研究表明，思维链自指导（CoT Self-Instruct）生成的问题和评分标准对弱求解器而言过于困难。如表 3 所示，弱求解器的平均得分仅为 0.159，许多尝试得分为 0，这阻碍了在群组相对策略优化（GRPO）下的学习。若将计算机科学场景中的硬阈值接受标准应用于此，大多数思维链数据点将被接受。相反，我们提出疑问：能否直接验证数据点的质量及其对 GRPO 的适用性？

**表 3 生成的法律推理任务的质量统计。**思维链自指导（CoT Self-Instruct）是标准提示生成；智能体自指导（Agentic Self-Instruct）是智能体循环的输出。两列均由 Kimi-K2.6 在生成时基于相同的 4B 弱求解器 / 397B 强求解器对进行评分。

| Metric                  | CoT Self-Instruct | Agentic Self-Instruct |
|-------------------------|-------------------|-----------------------|
| Weak solver avg         | 0.159             | 0.283                 |
| Strong solver avg       | 0.717             | 0.698                 |
| Gap (strong – weak)     | 0.558             | 0.415                 |
| Agentic rounds          | 1.00              | 4.98                  |
| Question length (chars) | 1,569             | 900                   |
| Rubric items            | 18.6              | 17.3                  |
| Weak rollout std        | 7.93              | 12.63                 |

**流程概述。**与我们在计算机科学论文任务中采用的硬编码接受标准的主要区别在于，此处我们采用更灵活的循环裁判来决定是否接受某一轮的生成结果。具体而言，每份法律文件首先通过一个专门的提取器智能体，生成结构化摘要（主题关键词、显著事实、裁判要点）。随后，挑战者智能体根据该摘要生成一个现实的法律问题、一个加权评分标准以及一个声明的目标能力集。每个候选项由弱求解器执行 5 次，由强求解器执行 3 次。

随后，裁判读取每次展开的求解器模式、弱 / 强差距以及评分标准，并返回结构化判定 (weak\_pattern,strong\_pattern,gap\_interpretation,rubric\_concerns,grpo\_suitability) 以及接受 / 改进决策。对于改进决策情形，裁判向挑战者提供具体的 suggestion\_for\_challenger（例如，“弱展开都在背诵相同的模板；将问题推向对判决要点的逐步应用，而非简单回忆”），随后循环重新运行；若接受，则该示例被视为质量合格，循环结束。与硬编码的计算机科学质量决策不同，用于法律任务的裁判没有固定的接受阈值：它根据对结果的分析（包括弱展开方差、差距以及 PRBench 基线求解器统计量）逐轮做出决定。优质的法律训练示例由整体数据质量和 GRPO 适用性共同定义，而不仅仅取决于数值差距目标。

智能体系统提示词见附录 C.2 小节。

**数据设置。**我们处理了 7.8 千份源文档，其中 5.7 千份生成了可用的思维链自指导示例，2.8 千份在智能体循环后达到接受判定。在第 3.2.2 小节的受控头对头强化学习比较中，智能体实验使用全部 2.8 千个已接受数据点，思维链自指导集使用从 5.7 千个思维链池中随机抽取的 2.8 千个示例。

### 3.2.1 智能体自指导循环分析

智能体自指导循环重塑了弱展开分布。表 3 展示了前后对比分析：在相同的源文档上，弱 / 强差距在智能体处理后实际上从 55.8 分缩小至 41.5 分。

表 4 在 PRBench 上评估的强化学习训练结果。自动数据智能体自指导方法优于使用标准思维链自指导创建数据的方法。我们使用 GRPO 在来自每个数据源的 2.8 千个法律问答对上训练 Qwen3.5-4B，并在 PRBench-Legal（500 个提示）和 PRBench-Legal-Hard（250 个提示子集）测试集上进行评估。分数为按提示裁剪的 PRBench 分数，由 Kimi-K2.6 和 GPT-5 共同评分。两位评分者一致认为：在智能体数据上训练的 4B 模型优于在标准思维链自指导数据上训练的模型，并且在两个划分上均优于大得多的 397B 基线模型。

| Response Model                                     | GPT-5 Grader |              | Kimi-K2.6 Grader |              |
|----------------------------------------------------|--------------|--------------|------------------|--------------|
|                                                    | Legal        | Legal-Hard   | Legal            | Legal-Hard   |
| Qwen3.5-4B (no additional RL)                      | 0.280        | 0.167        | 0.245            | 0.145        |
| Qwen3.5-397B (no additional RL)                    | 0.404        | 0.277        | 0.358            | 0.226        |
| Qwen3.5-4B RL on CoT Self-Instruct data            | 0.377        | 0.253        | 0.343            | 0.233        |
| <b>Qwen3.5-4B RL on Agentic Self-Instruct data</b> | <b>0.441</b> | <b>0.315</b> | <b>0.393</b>     | <b>0.266</b> |

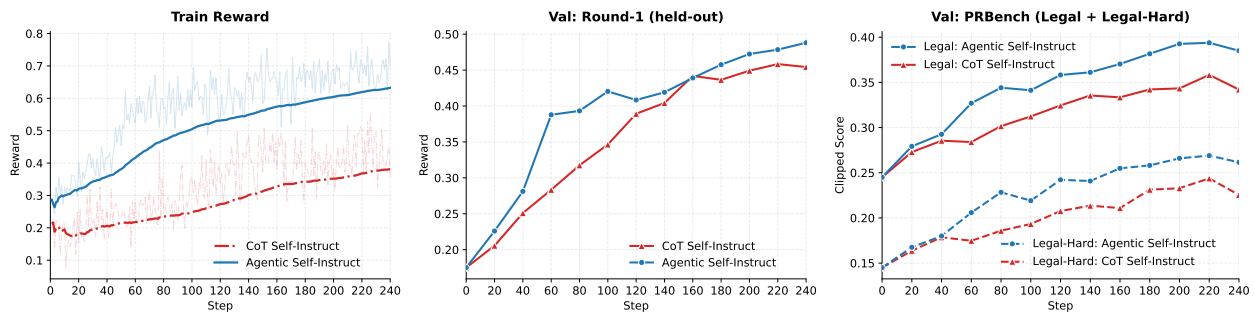


图 5 法律推理的强化学习训练动态。我们使用来自每个数据源（智能体自指导、思维链自指导）的 2.8k 法律问题-评分标准对，通过 GRPO 训练 Qwen3.5-4B，并每 20 步在 100 个提示的留出思维链集（中）以及 PRBench 法律/法律困难分割（右）上进行评估。所有奖励和分数均由 Kimi-K2.6 评分。在整个训练过程中，智能体方法在所有度量上均领先于思维链方法。

循环，与计算机科学设置不同。显著的变化在于每个提示的弱 rollout 标准差从 7.93 上升到 12.63。思维链问题将弱分数集中在零附近（均值 15.9%，中位数 10.7%）；许多提示的五个弱 rollout 中有四个或五个得分为零，这使得每组 GRPO 优势接近于零，提供的学习信号很少。智能体循环将弱均值推高到 28.3%，而强均值基本不变（71.7% → 69.8%）；同样的差距现在分布在可用的方差范围内。该循环通过重塑每个提示的奖励信号，使问题更易于学习。作为副产品，循环评判器的文本反馈还推动挑战者生成更短、应用风格的问题（平均 900 字符对比 1.6k 字符），而不改变评分标准密度，这偶然使格式与相对较短的 PRBench 法律提示对齐。

在每一轮中，循环评判器根据 rollout 模式给出一个分类 grpo\_suitability 判定（高 / 中 / 低）。在思维链自指导池中，分布为 4.8% 高/41% 中/45% 低；在智能体池中，分布为 52% 高/43% 中/2% 低。被接受问题的中位轮数为 4 轮智能体循环（均值 4.98，最大 19），仅 ~2% 使用单轮。

### 3.2.2 强化学习训练结果

我们使用 GRPO 对 Qwen3.5-4B 进行强化学习训练，设置如下：2.8k 智能体自指导提示（智能体）对比 2.8k 标准思维链自指导提示（思维链），每个提示  $n=8$  个 rollout，并使用 Kimi-K2.6 评分标准评判器作为训练时奖励。我们在 PRBench 法律（500 个提示，其中 250 个在 PRBench 官方评判器设置下被标记为 legal\_hard）（例如，评判器分别评估每个评分标准）上进行评估，并额外使用 GPT-5 作为独立、更强的评分器对相同的 rollout 重新评分，以确认比较结果不受 Kimi 评分器偏差影响。

在 500 个提示的 PRBench 法律分割上（表 4），在智能体数据上强化学习训练的 Qwen3.5-4B 得分为 0.441（GPT-5

作为裁判)和 0.393 (Kimi 作为裁判), 优于同架构的思维链训练模型 (0.377 / 0.343), 甚至优于未进行额外强化学习的更大规模强基线 Qwen3.5-397B-A17B (0.404 / 0.358)。在 PRBench-Legal-Hard 上保持了相同的排序。+0.05 - 0.06 智能体方法相对于思维链方法的优势是在相同的 2.8k 提示预算、相同的挑战者、相同的源语料库上获得的: 两种设置之间的唯一区别是训练数据创建中的智能体循环。图 5 中的训练曲线显示, 在我们评估的每个检查点上, 智能体方法在训练奖励、留出的思维链验证集以及 PRBench-Legal 上均处于领先地位。

### “更具挑战性”与“恰到好处”

我们在第 3.1 小节和第 3.2 小节中迄今为止实验的两个任务, 将相同的智能体自指导循环应用于标准思维链自指导基于提示的生成的不同失败模式: 在计算机科学中, 思维链问题对于弱求解器来说过于简单 (差距 0.02, 引发了对问题不够具有挑战性的担忧), 而在法律领域, 它们过于困难, 许多展开的得分为 0 (差距 0.56, 但提供了过于严苛的学习信号)。应用自动数据智能体循环后, 差距向相反方向移动 (在计算机科学中扩大, 在法律中缩小), 然而下游强化学习的结果是相同的: 在自动数据生成的数据上训练的模型在每个留出测试上都优于在思维链数据上训练的模型, 并且在 PRBench-Legal 上, 一个 4B 模型优于一个更大的基线。关键不是让问题更具挑战性, 而是让它们恰到好处, 以便模型能够进行爬山式学习; 智能体自指导循环正是让我们实现这一点的原因。

## 3.3 科学推理

接下来, 我们考虑构建具有挑战性的问题, 这些问题需要对与现有《原理》集合 (Aggarwal 等人, 2026) 相同类别和领域中的数学对象进行推理。《原理》集合是使用基于思维链自指导的方法 (提示式、多步骤大语言模型工作流) 设计的, 涵盖了来自 MSC2020 和 PHYS 目录的广泛课程。另一方面, 《原理》基准由现有数学和物理基准的人工标注子集组成, 其中问题经过筛选, 仅包含答案中涉及数学对象的问题。

**流程概述** 在本实验中, 我们使用弱求解器 Qwen3.5-4B 和强求解器 Qwen3.5-397B-A17B。弱求解器实际上是一个能力很强的推理模型, 能够解决《原理》集合中的许多问题。因此, 它是我们流程中一个很好的候选模型, 我们试图生成对弱求解器更具挑战性的问题。主协调智能体和挑战者是 Kimi K2.6。智能体系统提示在附录 C.3 小节中提供。构建的智能体数据中问题类型的详细分解在附录 B 中提供。

### 3.3.1 强化学习训练结果

该设置允许我们比较用于下游强化学习训练的三种数据源: (i) 思维链自指导: 直接在《原理》集合中的问题上进行训练, 这些问题也用作智能体自指导中的基础上下文; (ii) 智能体: 在智能体自指导生成的数据上进行训练; (iii) 组合: 在两种数据源上一起训练, 使训练集大小加倍。每个单独的数据源包含 9k 个训练示例和 1k 个留出评估示例, 而组合设置使用 18k 个训练示例。与之前的实验类似, 我们使用 Kimi K2.6 作为评判器来训练 Qwen3.5-4B 模型, 将模型生成的答案与参考答案进行比较, 并根据比较结果分配二元奖励。我们使用组大小为 8、批大小为 64 的 GRPO 进行训练。

我们使用组合验证集评估模型在训练分布内外的表现, 该验证集由来自智能体生成和思维链数据分布的留出示例以及分布外的《原理》基准组成。结果见表 5 和表 6。

在组合验证集上 (表 5), 在智能体自指导数据上训练带来了最大的整体改进 (平均@8 提升 +3.20%), 优于直接在思维链自指导数据上训练 (+2.42%) 和组合数据集 (+2.70%)。一个关键发现是, 智能体自指导数据即使在思维链验证子集上也提高了性能 (+3.05% 对比思维链自指导的 +1.86%), 尽管并未针对该分布进行显式优化。这表明在更难的问题上训练可以迁移到更简单的问题上:

表 5 在科学推理任务上评估的强化学习训练结果。智能体自指导数据优于思维链自指导甚至组合数据（2× 训练规模）。增量 (Δ) 是相对于起始 Qwen3.5-4B 模型而言的。

| Eval Subset              | Base       | CoT Self-Instruct |       | Agentic Self-Instruct |              | Combined (2× data) |              |
|--------------------------|------------|-------------------|-------|-----------------------|--------------|--------------------|--------------|
|                          | Qwen3.5-4B | avg@8             | Δ     | avg@8                 | Δ            | avg@8              | Δ            |
| <b>Overall</b>           | 68.66%     | 71.08%            | +2.42 | <b>71.86%</b>         | <b>+3.20</b> | 71.36%             | +2.70        |
| Agentic subset           | 52.39%     | 56.33%            | +3.94 | <b>56.79%</b>         | <b>+4.40</b> | 55.88%             | +3.49        |
| CoT Self-Instruct subset | 77.17%     | 79.03%            | +1.86 | <b>80.22%</b>         | <b>+3.05</b> | 79.66%             | +2.49        |
|                          |            | pass@8            | Δ     | pass@8                | Δ            | pass@8             | Δ            |
| <b>Overall</b>           | 87.73%     | 88.58%            | +0.85 | <b>88.91%</b>         | <b>+1.18</b> | 88.75%             | +1.02        |
| Agentic subset           | 81.13%     | 81.74%            | +0.61 | 81.86%                | +0.73        | <b>82.11%</b>      | <b>+0.98</b> |
| CoT Self-Instruct subset | 92.52%     | 92.77%            | +0.25 | <b>94.36%</b>         | <b>+1.84</b> | 93.50%             | +0.98        |

表 6 分布外基准测试结果，比较不同训练数据来源。尽管智能体自指导数据仅使用了组合数据一半的数据量，却带来了最大的整体提升。

| Category       | Items | Base       | CoT Self-Instruct |              | Agentic Self-Instruct |              | Combined (2× data) |              |
|----------------|-------|------------|-------------------|--------------|-----------------------|--------------|--------------------|--------------|
|                |       | Qwen3.5-4B | avg@8             | Δ            | avg@8                 | Δ            | avg@8              | Δ            |
| <b>Overall</b> | 2113  | 50.43%     | 51.10%            | +0.67        | <b>51.47%</b>         | <b>+1.04</b> | 51.17%             | +0.74        |
| ARB            | 47    | 81.91%     | <b>83.78%</b>     | <b>+1.87</b> | 80.32%                | −1.59        | 83.24%             | +1.33        |
| Physics        | 110   | 66.22%     | 65.91%            | −0.31        | <b>67.05%</b>         | <b>+0.83</b> | 66.25%             | +0.03        |
| RealMath       | 632   | 33.68%     | 35.25%            | +1.57        | <b>35.43%</b>         | <b>+1.75</b> | 34.97%             | +1.29        |
| SuperGPQA      | 1324  | 56.00%     | 56.28%            | +0.28        | <b>56.82%</b>         | <b>+0.82</b> | 56.50%             | +0.50        |
|                |       |            | pass@8            | Δ            | pass@8                | Δ            | pass@8             | Δ            |
| <b>Overall</b> | 2113  | 59.54%     | <b>60.06%</b>     | <b>+0.52</b> | 59.91%                | +0.37        | 59.91%             | +0.37        |
| ARB            | 47    | 91.49%     | 91.49%            | +0.00        | 91.49%                | +0.00        | <b>93.62%</b>      | <b>+2.13</b> |
| Physics        | 110   | 76.36%     | 77.27%            | +0.91        | <b>79.09%</b>         | <b>+2.73</b> | 72.73%             | −3.63        |
| RealMath       | 632   | 41.93%     | 43.67%            | +1.74        | 43.67%                | +1.74        | <b>44.30%</b>      | <b>+2.37</b> |
| SuperGPQA      | 1324  | 65.41%     | 65.33%            | −0.08        | 64.95%                | −0.46        | 65.11%             | −0.30        |

我们迭代式智能体流程所产生的具有挑战性的示例，所教授的推理技能能够泛化到超出其特定难度目标的范围。

在分布外基准测试（表 6）中，智能体自指导再次取得了最佳的整体平均@8 提升（+1.04%），在大多数类别上均有稳定提升，尤其是在真实数学（+1.75%）和超级研究生水平问答（+0.82%）上。这种迁移效应进一步证实，智能体自指导生成的更困难问题能够构建更稳健的推理能力。

通过@8 的结果揭示了各方法之间更为细致的权衡。组合数据在多个类别的通过@8 上显示出优势：抽象推理基准（+2.13%，对比智能体和基础均为+0.00%）和真实数学（+2.37%，对比智能体的+1.74%）。这表明，虽然智能体自指导通过教会模型更可靠地解决挑战性问题来提升平均性能，但组合数据更大的多样性（和规模）可能帮助模型偶尔解决更广泛的问题（反映在更高的通过@8 上）。一种可能的解释是，千问 3.5-4B 可能正在接近其在该任务分布上的容量极限，而更大的模型可能更好地利用组合数据，同时在这两个指标上取得提升。

这些结果凸显了数据质量和难度的重要性。智能体自指导中的迭代过程产生了具有挑战性的示例，提供了更高效的学习信号——在这些更困难的问题上进行训练，不仅提升了在困难任务上的性能，还迁移到了更简单的任务上。这支持了以下假设：将推理时的计算资源投入到生成更高质量、更具挑战性的合成数据中，可能比单纯扩大数据集规模更有效。此外，我们发现训练显著降低了推理截断率（对于智能体自指导，在 65,536 个标记的预算下，从 23.75% 降至 4.09%），其中大约一半的准确率提升可归因于更具标记效率的推理（见附录 A）。

## Weak-Strong Challenger: Evolve Architecture

optimizes QA generation prompts to separate weak (4B) from strong (397B) models

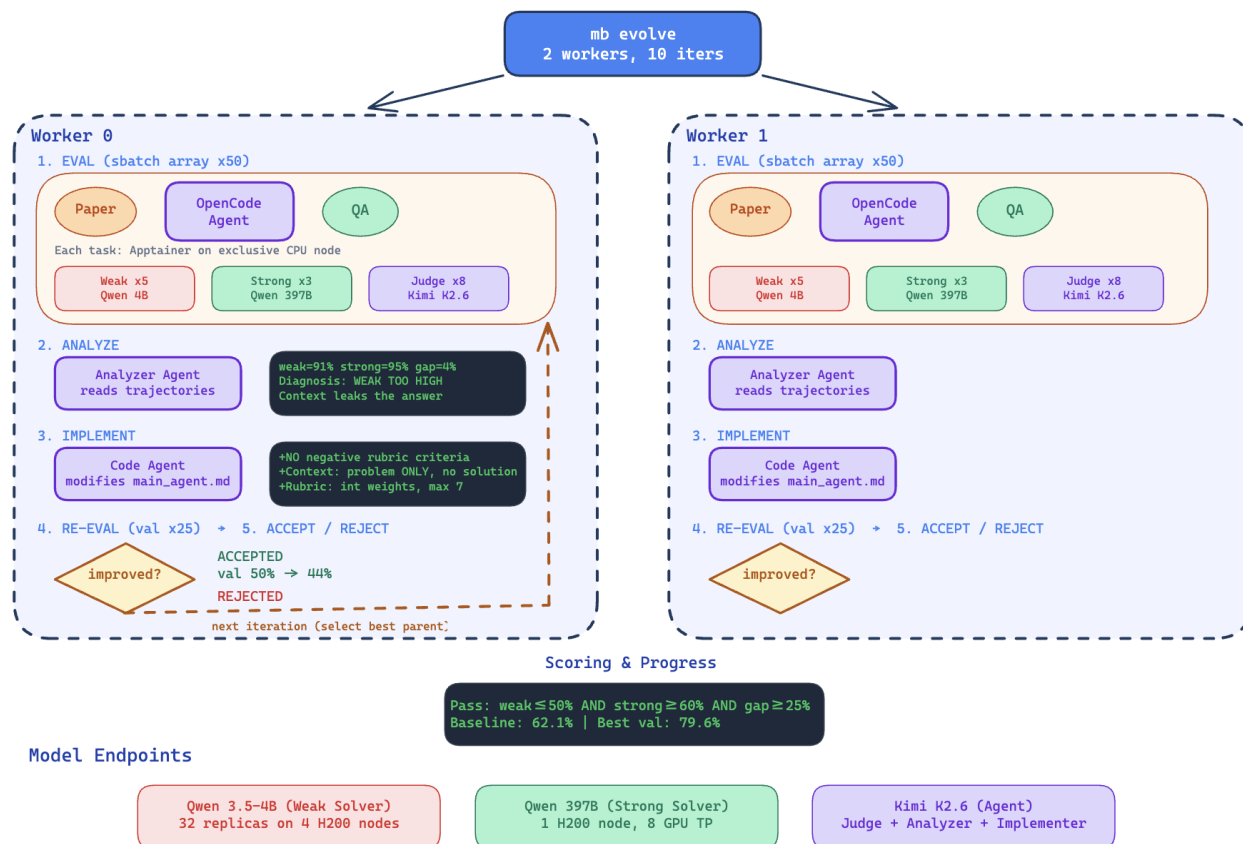


图 6 数据科学家智能体的元优化。外层优化循环在计算机科学论文任务的训练样例上评估智能体的提示词，分析失败轨迹以识别系统性弱点（例如上下文泄露），通过代码编辑智能体实施提示词修改，并在留下的验证论文上重新评估。只有当修改提高了弱-强分离率时，修改才会被接受。该过程在 233 次总迭代中接受了 126 次迭代，将验证通过率从 62.1% 提高到 79.6%。

## 4 数据科学家的元优化

到目前为止，我们使用固定的智能体自指导框架实现了自动数据智能体，并提供了定义智能体策略的提示词。然而，也可以对整体智能体进行（元）学习。因此，在本节中，我们将元优化应用于数据科学家智能体本身，使用内层循环的相同评估标准来指导外层循环的优化——即智能体的提示词和策略。具体来说，我们使用一个进化优化框架，将智能体的脚手架视为需要迭代改进的代码。

方法。元优化器维护一个候选提示词种群，每个提示词由相对于基线仓库的代码差异定义。每次迭代按以下步骤进行：（1）通过玻尔兹曼采样从种群中选择一个父代，其中候选  $c$  被选中的概率与  $\exp(score_c/T)$  成正比，温度为  $T=0.1$ ，在保持探索的同时强烈偏向高分候选；（2）在一小批训练论文上评估父代的提示词，收集智能体轨迹和弱/强求解器得分；（3）使用一个大型语言模型智能体分析轨迹，该智能体读取完整的求解器交流并撰写系统性失败模式的根因分析；（4）通过代码编辑智能体实施提示词修改，该智能体读取分析、迭代历史和当前提示词，然后生成改进的差异；（5）重新评估

在留出验证论文上同时评估父代与变异体；（6）接受或拒绝该变异体——仅当其验证分数严格超过其父代时，才将其加入种群；（7）将结果总结为历史日志，供后续分析器读取。多次迭代并发运行，父代选择相互独立。由于单次评估分数存在噪声（求解器在温度 1.0 下运行），被接受的候选者在后续被采样为父代时会累积额外评估，我们报告所有重新评估的平均分数。

设置。我们对第 3.1 小节中的计算机科学研究论文任务进行元优化。元优化器使用 Kimi-K2.6 同时作为分析器（读取评估轨迹以诊断失败模式）和实现器（修改智能体的提示词）。被优化的内循环智能体同样使用 Kimi-K2.6，采用多智能体配置，包含独立的挑战者、主智能体和质量验证器提示词，遵循第 3.1 小节。我们使用 50 篇训练论文和 25 篇验证论文。生成的问答对若满足以下所有标准则视为成功：弱求解器（Qwen3.5-4B）得分  $\leq 65\%$ ，弱求解器最佳尝试得分  $\leq 75\%$ ，强求解器（Qwen3.5-397B-A17B）得分  $\geq 60\%$  且  $\leq 95\%$ ，强求解器与弱求解器之间的差距  $\geq 20$  个百分点，由基于评分标准的评估判定。

结果。从在 100 个验证样本上达到  $\sim 62.1\%$  验证问答通过率的基线提示词开始，元优化器在 233 次迭代中逐步发现提示词改进。我们报告在每次会话 6 小时超时下的基线和最终进化框架结果。结果见表 7。

**表 7 数据科学家智能体在计算机科学研究论文任务上的元优化。**优化器迭代改进智能体的提示词，每次被接受的迭代都建立在前一个最佳版本之上。验证通过率（重新评估）衡量生成的问答对成功区分弱求解器和强求解器的比例，并在多次重新评估中取平均以减少噪声。

| Meta Iteration | Num Val Sample | Val Pass Rate 6h |
|----------------|----------------|------------------|
| Baseline       | 100            | 62.1%            |
| Iter 124       | 100            | 79.6%            |

元优化器通过轨迹分析识别出若干系统性失败模式——检查弱求解器在其响应中实际说了什么，并确定通用答案和评分标准格式错误是区分度差的主要原因。优化器通过以下提示词修改来解决这些问题，这些修改是在 233 次迭代过程中自动发现的：

- **论文特定洞见强化：**优化器新增指令，要求问题必须测试论文特有的知识，而非通用的机器学习或计算机科学知识。引入了一项自检机制：“若解题者无需阅读该特定论文即可正确作答，则问题过于简单。”这一机制直接解决了弱解题者通过生成看似合理的通用回答而获得高分的问题。
- **上下文泄露预防：**新增严格规则，要求上下文仅描述问题领域与设定，绝不涉及论文提出的解决方案。引入了一项自检机制：“能否通过改写上下文中的句子来回答问题？若能，则需重写。”
- **仅含正权重且权重封顶的评分标准：**优化器移除了负权重的评分标准项，发现这些项在历史上曾误触发并严重损害强模型的得分，且未能提升区分度。相反，所有标准项均采用正整数权重，且上限为 7，防止任何单一标准主导总分。这是一项反直觉的发现——理论上惩罚错误似乎有益，但在实践中却造成了损害。
- **结构化评分标准格式：**优化器强制评分标准项采用严格的 JSON 格式，并使用整数权重，消除了早期迭代中导致评估失败的解析错误（例如，字符串权重“+8”而非整数 8）。

验证通过率从 62.1% 提升至 79.6% 表明，对数据科学家智能体的指令进行元优化能够在不依赖人工提示工程的情况下显著提升数据质量，尽管绝对数值也凸显了可靠生成能够区分不同能力水平模型的问题的难度。

## 5 相关工作

合成指令与对齐数据。合成数据已成为语言模型后训练的核心要素，尤其是在人工编写的监督数据变得昂贵、稀缺或缺乏足够挑战性的情况下。自指令（Self-Instruct）方法（Wang 等人，2023 年）提出了一种简单且有影响力的方案，即语言模型从一个小型种子集合中引导生成新的指令遵循示例。后续工作从多个方向扩展并丰富了这一思路：从更强的教师模型进行指令蒸馏（Mukherjee 等人，2023 年）、大规模合成对话（Ding 等人，2023 年）、人工智能生成的偏好与反馈数据（Cui 等人，2023 年），以及自动指令进化以提升任务复杂度和多样性（Xu 等人，2024 年）。更近期的工作如喜鹊（Magpie）（Xu 等人，2025 年）进一步表明，对齐后的大语言模型（LLM）能够通过极简的提示合成大规模对齐数据。这些方法确立了大语言模型作为强大数据生成器的地位，但通常将生成视为一个基本固定的提示或过滤流程。相比之下，自动数据（Autodata）将数据创建视为一个迭代的数据科学过程：智能体生成示例、评估其效用、分析失败原因，并修订其数据生成方案。

基于事实依据、可验证且以推理为核心的合成数据。另一条研究路线强调，合成数据的质量在很大程度上取决于事实依据、领域特异性（如特定的可验证任务）以及推理迹的形式。在代码和数学推理领域，合成的“教科书”式数据和练习在训练小型但性能强大的模型方面发挥了关键作用（Li 等，2023）。MetaMath（Yu 等，2024）、MAmmoTH（Yue 等，2024）和 OpenMathInstruct（Toshniwal 等，2025）表明，合成或半合成的数学推理数据能够显著提升下游问题求解能力。基于事实依据的方法，如 Source2Synth（Lupidi 等，2024）和 NaturalReasoning（Yuan 等，2025），从真实文档或表格中生成示例，并针对可回答性进行筛选；而 CoT-Self-Instruct（Yu 等，2025）则利用思维链规划和过滤来改进合成数据，使其同时适用于可验证推理和开放式指令遵循。Autodata 在这些基于事实依据且具有推理感知能力的数据生成方法基础上构建，但增加了一个显式的智能体循环，利用求解器行为和评估器反馈来使生成的数据适应目标模型。

智能体数据生成与自动化数据科学系统。近期若干系统从单提示生成转向智能体数据或数据科学工作流。AgentInstruct（Mitra 等，2024）在精神上接近我们的工作：它利用智能体流程生成大规模、多样化的合成后训练数据。我们的工作则将数据创建视为一个迭代的数据科学循环，其中智能体生成示例、评估其学习效用、分析失败并修订其配方。特别是，智能体自指导（Agentic Self-Instruct）利用弱-强求解器行为和评判反馈来调整数据难度，并可进一步对数据科学家智能体本身进行元优化。Khan 等（2025）也将数据生成表述为一个序贯决策问题，其中智能体观察学生模型的错误与成功，以生成有针对性的训练数据来改进学生。此外，已有工作使用“AutoData”这一名称（Ma 等，2026），研究了一个用于开放网络数据收集的多智能体系统，利用专门智能体根据自然语言指令收集数据集，这也可视为我们框架的一个相关特例。与此同时，基于大语言模型的数据科学智能体，如 DS-Agent（Guo 等，2024）和 Data Interpreter（Hong 等，2025），自动化了数据科学工作流的部分环节，包括规划、编码、模型训练、调试和分析。这些工作表明大语言模型智能体能够执行复杂的数据导向工作流。我们的工作目标不同：该智能体主要不是收集网络数据或解决数据科学竞赛，而是充当一名数据科学家，其输出是用于另一模型的训练或评估数据。因此，核心优化目标是所生成数据的学习价值，由任务特定评估器和求解器行为来衡量。

自我改进、自我博弈与挑战者-求解者数据。自动数据也与自我改进和自我博弈方法相关，在这些方法中，模型为自身训练生成数据、奖励或任务。STaR（Zelikman 等，2022）通过迭代生成并训练成功的推理依据来引导推理迹。自奖励语言模型（Yuan 等，2024）在迭代偏好训练期间使用模型自身作为大模型作为评审的奖励来源。更具对抗性或课程导向的方法利用自生成任务训练模型：自挑战语言模型智能体（Zhou 等，2025）生成工具使用任务及验证函数，绝对零度（Zhao 等，2025a）提出并求解其自身可验证的

推理任务无需外部数据，SPICE (Liu 等, 2025) 采用基于语料的挑战者-推理者框架。本文的弱-强智能体自指导实例化同样采用挑战者为求解器创建任务的思想，但将其置于更广泛的数据科学家循环中：智能体分析求解器失败原因，判断示例质量，调整难度，并优化对学习有用的示例，而非仅仅追求难度。

大语言模型评判器、过滤与数据选择。大量合成数据工作依赖于生成后的过滤、评判或选择示例。自指令 (Self-Instruct) 采用启发式过滤剔除无效或近似重复的示例 (Wang 等, 2023)；巫师大语言模型 (WizardLM) 及相关演化方法提升了复杂度，但仍需数据质量控制 (Xu 等, 2024)；超反馈 (UltraFeedback) 扩展了人工智能反馈以支持偏好学习 (Cui 等, 2023)；思维链自指令 (CoT-Self-Instruct) 则利用答案一致性或基于奖励模型的过滤来挑选高质量示例 (Yu 等, 2025)。自动数据 (Autodata) 更积极地使用此类评估信号。评判器的反馈并非用于过滤静态池，而是融入生成循环本身。在本文实验中，这一区别至关重要：在计算机科学研究任务中，智能体使示例更难且更具区分度；而在法律推理任务中，智能体通过避免全零弱展开，使示例退化程度更低、更适配群体相对策略优化 (GRPO)。因此，目标并非单纯追求高质量或高难度，而是为靶模型提供有效学习信号的数据。

智能体支架的自动研究与优化。最后，自动数据 (Autodata) 与提示、支架及智能体框架的自动化研究与优化工作相衔接。提示优化方法如提示培育器 (Promptbreeder) (费尔南多等人, 2023)、自我精炼 (Self-Refine) (马丹等人, 2023)、作为优化器的大语言模型 (LLMs as Optimizers) (杨等人, 2024) 以及近期的引导式进化提示优化 (GEPA) (阿格拉瓦尔等人, 2025) 表明，大语言模型系统能够通过迭代反馈改进提示、解决方案或策略。此外，近期的自动研究系统旨在自动化科学循环的更大部分：人工智能科学家 (The AI Scientist) (卢等人, 2024) 执行构思、实验实施、结果分析、论文撰写和模拟评审，而自动研究 (autoresearch) (卡帕西, 2026) 探索修改训练代码和配方的智能体。元框架 (Meta-Harness) (李等人, 2026) 将大语言模型系统周围的框架视为端到端优化的对象。我们的元优化实验将这一视角应用于数据创建本身：外循环利用指导内层数据生成循环的相同数据质量标准，改进数据科学家智能体的提示和策略。

定位。先前工作已表明，大语言模型能够合成指令、对话、反馈、推理轨迹、领域特定数据集，甚至自我博弈任务。自动数据 (Autodata) 在明确的智能体数据科学表述下统一了这些思想。其关键区别在于，数据生成、评估、失败分析、配方修订和元优化都是循环的一部分。这提供了一种通用机制，可将更强的推理时模型和更大的智能体计算预算转化为更高质量的训练和评估数据。

## 6 结论与讨论

本文提出了自动数据 (Autodata) 框架，在该框架中，自主智能体扮演数据科学家的角色——生成合成数据，利用任务特定信号进行评估，并根据结果改进其数据生成方案。我们通过智能体自指导 (Agentic Self-Instruct) 实例化了这一思想，该方法显式优化能够区分弱求解器与强求解器的样本，并在计算机科学研究任务、法律推理任务以及数学对象推理中展示了一致的质量提升。最后，我们证明了数据科学家智能体本身可以被元优化，无需手动提示工程即可带来显著的额外改进。我们相信，本文报告的实验只是冰山一角，对该方法的进一步探索和优化将带来更多收益。我们在下文列出了未来方向。

更多任务、模型和基线。未来的持续工作应探索该方法在更多样化任务和模型上的应用。我们设想一个理想的系统是一个通用的自动数据智能体，可用于任何类型的数据（数学、代码、通用指令遵循任务、安全等），从可验证到不可验证，从单轮对话到多轮对话再到智能体任务。

破解与局限性。我们遇到了智能体试图避免正确完成工作或试图“欺骗”目标的实例，例如通过修改给弱求解器的提示，告诉它表现弱一些。我们已通过简单地在智能体流水线上施加更多约束来部分解决这一问题，但计划研究更强的防护措施，使智能体能够拥有比本文定义的刚性迭代循环更多的行动自由和工具使用权限。同样，我们希望确保数据既具有挑战性又有意义，例如在计算机科学任务中，我们发现一些生成的问题和评分标准过度依赖于论文中的具体实验数字，而不是测试可泛化的推理能力。

全数据集分析迭代。我们的初步实验在示例层面生成高质量数据。如第 2 节中自动数据（Autodata）的总体描述所述，我们希望将其扩展到数据集层面的分析，以提升质量，例如多样性统计以及在与现有数据集交互方式上的整体改进。介于完整数据集分析之间的中间步骤是迭代分批分析，即生成  $N$  个示例，然后从当前批次中提炼经验，用于生成下一批次。

从自我改进到协同改进。我们及其他人在自我博弈（Self-Play）方面的工作（Zhou 等，2025；Yuan 等，2024；Zhao 等，2025a；Liu 等，2025）也涉及构建一个“挑战者”，为求解器生成训练示例，该挑战者可以与奖励和权重更新一起优化，而非采用上述智能体方式。然而，完整的自我改进循环可以将我们的智能体自我指令系统视为挑战者，并在训练求解器的同时，对其学习技能和权重进行训练。在本工作中，我们探索了一种类似自动研究（Autoresearch）的方法（Karpathy，2026）来训练我们的智能体，但该方向仍有大量探索空间。最后，在当前完整的模型训练流程中，完全将人类从循环中移除可能并不理想，尤其是当数据创建对模型能力和安全行为如此重要时。纳入人类反馈以及与智能体进行“协同研究”的能力可能是更好的路径，称为协同改进（Co-improvement）（Weston 和 Foerster，2025），这是我们未来研究的主要方向。

## 参考文献

- Pranjal Aggarwal, Marjan Ghazvininejad, Seungone Kim, Ilia Kulikov, Jack Lanchantin, Xian Li, Tianjian Li, Bo Liu, Graham Neubig, Anaelia Ovalle, et al. Reasoning over mathematical objects: on-policy reward modeling and test time aggregation. *arXiv preprint arXiv:2603.18886*, 2026.
- Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. Gepa: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.
- Afra Feyza Akyürek, Advait Gosai, Chen Bo Calvin Zhang, Vipul Gupta, Jaehwan Jeong, Anisha Gunjal, Tahseen Rabbani, Maria Mazzone, David Randolph, Mohammad Mahmoudi Meymand, et al. Prbench: Large-scale expert rubrics for evaluating high-stakes professional reasoning. *arXiv preprint arXiv:2511.11562*, 2025.
- Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Qiang Yue, Yuan Ni, Guotong Xie, Zhiyuan Liu, and Maosong Sun. Ultrafeedback: Boosting language models with high-quality feedback. 2023.
- Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. Enhancing chat language models by scaling high-quality instructional conversations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3029–3051, 2023.
- Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. *arXiv preprint arXiv:2309.16797*, 2023.
- Siyuan Guo, Cheng Deng, Ying Wen, Hechang Chen, Yi Chang, and Jun Wang. Ds-agent: Automated data science by empowering large language models with case-based reasoning. *arXiv preprint arXiv:2402.17453*, 2024.
- Peter Henderson, Mark Krass, Lucia Zheng, Neel Guha, Christopher D Manning, Dan Jurafsky, and Daniel Ho. Pile of law: Learning responsible data filtering from the law and a 256gb open-source legal dataset. *Advances in Neural Information Processing Systems*, 35:29217–29234, 2022.
- Sirui Hong, Yizhang Lin, Bang Liu, Bangbang Liu, Binhao Wu, Ceyao Zhang, Danyang Li, Jiaqi Chen, Jiayi Zhang, Jinlin Wang, et al. Data interpreter: An llm agent for data science. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 19796–19821, 2025.
- Andrej Karpathy. autoresearch: Ai agents running research on single-gpu nanochat training automatically. <https://github.com/karpathy/autoresearch>, 2026. GitHub repository.
- Zaid Khan, Elias Stengel-Eskin, Jaemin Cho, and Mohit Bansal. Dataenvgym: Data generation agents in teacher environments with student feedback. In *International Conference on Learning Representations*, volume 2025, pages 53480–53507, 2025.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026.
- Yuanzhi Li, Sébastien Bubeck, Ronen Eldan, Allie Del Giorno, Suriya Gunasekar, and Yin Tat Lee. Textbooks are all you need ii: phi-1.5 technical report. *arXiv preprint arXiv:2309.05463*, 2023.
- Bo Liu, Chuanyang Jin, Seungone Kim, Weizhe Yuan, Wenting Zhao, Ilia Kulikov, Xian Li, Sainbayar Sukhbaatar, Jack Lanchantin, and Jason Weston. Spice: Self-play in corpus environments improves reasoning. *arXiv preprint arXiv:2510.24684*, 2025.
- Kyle Lo, Lucy Lu Wang, Mark Neumann, Rodney Kinney, and Daniel S Weld. S2orc: The semantic scholar open research corpus. In *Proceedings of the 58th annual meeting of the association for computational linguistics*, pages 4969–4983, 2020.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.
- Alisia Lupidi, Carlos Gemmell, Nicola Cancedda, Jane Dwivedi-Yu, Jason Weston, Jakob Foerster, Roberta Raileanu, and Maria Lomeli. Source2synth: Synthetic data generation and curation grounded in real data sources. *arXiv preprint arXiv:2409.08239*, 2024.
- Tianyi Ma, Yiyue Qian, Zheyuan Zhang, Zehong Wang, Xiaoye Qian, Feifan Bai, Yifan Ding, Xuwei Luo, Shinan Zhang, Keerthiram Murugesan, et al. Autodata: A multi-agent system for open web data collection. *Advances in Neural Information Processing Systems*, 38:173416–173448, 2026.

- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhunoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594, 2023.
- Arindam Mitra, Luciano Del Corro, Guoqing Zheng, Shweti Mahajan, Dany Rouhana, Andres Codash, Yadong Lu, Wei-ge Chen, Olga Vrousos, Corby Rosset, et al. Agentinstruct: Toward generative teaching with agentic flows. *arXiv preprint arXiv:2407.03502*, 2024.
- Subhabrata Mukherjee, Arindam Mitra, Ganesh Jawahar, Sahaj Agarwal, Hamid Palangi, and Ahmed Awadallah. Orca: Progressive learning from complex explanation traces of gpt-4. *arXiv preprint arXiv:2306.02707*, 2023.
- Vedant Shah, Dingli Yu, Kaifeng Lyu, Simon Park, Jiatong Yu, Yinghui He, Nan Rosemary Ke, Michael Mozer, Yoshua Bengio, Sanjeev Arora, et al. Ai-assisted generation of difficult math questions. *arXiv preprint arXiv:2407.21009*, 2024.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Shubham Toshniwal, Wei Du, Ivan Moshkov, Branislav Kisacanin, Alexan Ayrappetyan, and Igor Gitman. Openmathinstruct-2: Accelerating ai for math with massive open-source instruction data. In *International Conference on Learning Representations*, volume 2025, pages 19243–19275, 2025.
- Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*, pages 13484–13508, 2023.
- Jason Weston and Jakob Foerster. Ai & human co-improvement for safer co-superintelligence. *arXiv preprint arXiv:2512.05356*, 2025.
- Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, Qingwei Lin, and Daxin Jiang. Wizardlm: Empowering large pre-trained language models to follow complex instructions. In *International Conference on Learning Representations*, volume 2024, pages 30745–30766, 2024.
- Zhangchen Xu, Fengqing Jiang, Luyao Niu, Yuntian Deng, Radha Poovendran, Yejin Choi, and Bill Yuchen Lin. Magpie: Alignment data synthesis from scratch by prompting aligned llms with nothing. In *International Conference on Learning Representations*, volume 2025, pages 76346–76382, 2025.
- Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations*, volume 2024, pages 12028–12068, 2024.
- Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. Metamath: Bootstrap your own mathematical questions for large language models. In *International Conference on Learning Representations*, volume 2024, pages 45040–45061, 2024.
- Ping Yu, Jack Lanchantin, Tianlu Wang, Weizhe Yuan, Olga Golovneva, Ilia Kulikov, Sainbayar Sukhbaatar, Jason Weston, and Jing Xu. Cot-self-instruct: Building high-quality synthetic prompts for reasoning and non-reasoning tasks. *arXiv preprint arXiv:2507.23751*, 2025.
- Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason E Weston. Self-rewarding language models. In *Forty-first International Conference on Machine Learning*, 2024.
- Weizhe Yuan, Jane Yu, Song Jiang, Karthik Padthe, Yang Li, Ilia Kulikov, Kyunghyun Cho, Dong Wang, Yuandong Tian, Jason E Weston, et al. Naturalreasoning: Reasoning in the wild with 2.8 m challenging questions. *arXiv preprint arXiv:2502.13124*, 2025.
- Xiang Yue, Xingwei Qu, Ge Zhang, Yao Fu, Wenhao Huang, Huan Sun, Yu Su, and Wenhui Chen. Mammoth: Building math generalist models through hybrid instruction tuning. In *International Conference on Learning Representations*, volume 2024, pages 40320–40341, 2024.
- Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. Star: Bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems*, 35:15476–15488, 2022.
- Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv preprint arXiv:2505.03335*, 2025a.

Wenting Zhao, Pranjal Aggarwal, Swarnadeep Saha, Asli Celikyilmaz, Jason Weston, and Ilia Kulikov. The majority is not always right: RL training for solution aggregation. *arXiv preprint arXiv:2509.06870*, 2025b.

Yifei Zhou, Sergey Levine, Jason Weston, Xian Li, and Sainbayar Sukhbaatar. Self-challenging language model agents. *arXiv preprint arXiv:2506.01716*, 2025.

## A 主实验中的令牌效率与截断

我们通过考察截断率（响应中 `finish_reason=length`）来分析训练对令牌效率的影响，并将准确率提升归因于截断减少与推理改进。

### A.1 截断率

在实验中，本文将推理令牌预算设定为 65,536 个令牌。表 8 展示了不同训练配置下的截断率。基础千问 3.5-4B 模型表现出较高的截断率（组合验证集上为 23.75%，原理基准上为 17.06%），表明许多响应在模型完成推理之前就超出了这一宽松的 65K 令牌预算。训练显著降低了截断率：智能体自指导将截断率分别降至 4.09% 和 1.85%，而基础化方法则达到 10.00% 和 6.62%。这表明训练使模型能够在令牌预算内更简洁、高效地进行推理。

表 8 在 65,536 个令牌推理预算下，不同训练配置的截断率 (`finish_reason=length`)。训练显著提高了令牌效率，其中智能体自指导实现了最低的截断率。

| Model             | Combined-Val | Principia Bench |
|-------------------|--------------|-----------------|
| Qwen3.5-4B (base) | 23.75%       | 17.06%          |
| + Grounding       | 10.00%       | 6.62%           |
| + Agentic         | 4.09%        | 1.85%           |
| + Combined        | 3.37%        | 1.67%           |

### A.2 准确率提升的归因分析

为了解准确率提升的来源，本文对智能体验证子集（816 个问答项 × 8 次生成 = 6,528 个配对生成）进行了归因分析。对于每个从错误（基础模型）翻转为正确（训练后模型）的生成，本文将改进归为三类来源：

- **截断修复**：基础模型被截断，而训练后模型成功完成推理。
- **非截断推理**：两者均未被截断，但训练后模型推理正确。
- **其他**：剩余情况（例如，两者均被截断但训练后的模型仍然正确）。

结果如表 9 所示。在所有训练配置中，约 50% 的准确率提升来自修复截断问题。对于智能体自指导（Agentic Self-Instruct）数据，945 个翻转生成中有 54.81% 归因于截断修复，41.06% 归因于对未截断示例推理能力的提升。这表明，在 65K 令牌预算内学会高效推理是性能提升的主要因素之一，同时推理质量本身也有所改善。

表 9 智能体验证子集上准确率提升的归因分析。 $\Delta\text{Acc}$ : 相对于基础模型的准确率变化。 $\Delta\text{Trunc}$ : 截断率变化。截断修复占比表示错误 → 正确翻转中可归因于解决截断问题的比例。

| Model       | Accuracy | $\Delta\text{Acc}$ | Trunc. | $\Delta\text{Trunc}$ | Incorrect → Correct Attribution |              |              |
|-------------|----------|--------------------|--------|----------------------|---------------------------------|--------------|--------------|
|             |          |                    |        |                      | Trunc-fixed                     | Non-trunc    | Other        |
| + Agentic   | 56.79%   | +4.84              | 6.43%  | -27.65               | 518 (54.81%)                    | 388 (41.06%) | 39 (4.13%)   |
| + Grounding | 56.33%   | +4.38              | 15.82% | -18.26               | 441 (47.83%)                    | 367 (39.80%) | 114 (12.36%) |
| + Combined  | 55.88%   | +3.94              | 5.06%  | -29.03               | 511 (54.71%)                    | 390 (41.76%) | 33 (3.53%)   |

启示。这些发现表明，像 Qwen3.5-4B 这样的长推理模型经常失败，并非因为缺乏推理能力，而是因为在完成思维链之前就用尽了令牌——即使有 65,536 个令牌的充裕预算。在具有挑战性的数据上进行训练——尤其是智能体自指导数据——教会模型更简洁地推理，从而将冗长的推理模式转化为高效的模式。

这凸显了合成数据训练一个被低估的好处：除了提升推理质量外，它还能提高推理效率，使模型能够在固定的计算预算内解决更多问题。

## B 基于原理的智能体自指导数据的问题类型分析

### B.1 标注流程

为了刻画生成问题的推理需求，我们从完整的智能体数据中随机抽取了 1,000 个经过验证的问答对进行标注。我们采用了两阶段基于大语言模型的标注流程（Kimi-K2.6）：

1. **分类体系发现**。我们按挑战分数分层抽样了 200 个项目，以每批 20 个的方式呈现给模型，并要求其提出带有定义和示例的问题类型类别。98 条原始提议被整合为 11 种互不重叠的类型。
2. **标注**。使用发现的分类体系，将 1,000 个项目中的每一个都归入 11 种类型中的恰好一种。在过滤解析失败后，687 个项目获得了有效标注。

### B.2 问题类型

我们将这 11 种类型组织为三个类别。

**推理**——需要多步推导、分析或证明的问题：

- **多步符号与解析推导**。通过链式代数或微积分操作，从给定模型推导出闭式表达式。
- **组合、离散与结构分析**。计数配置、分析有限结构或确定组合不变量。
- **概率、随机与动力系统分析**。求解概率模型或动力系统，以获得分布、期望或稳态。
- **谱、稳定性、特征值与优化分析**。寻找特征值、临界点或极值；通过特征方程分析稳定性。
- **渐近、标度与微扰分析**。在极端条件下提取极限行为、幂律标度或微扰修正。
- **证明、形式化论证与验证**。构造严格论证以证明或反驳某一论断。

**知识**——可通过回忆或直接套用公式回答的问题：

- **直接公式、恒等式与定理应用**。选择已知公式或定理，并在一两步内代入参数。
- **事实性与定义性回忆**。检索既定事实、常数或定义，无需推导。

**混合型**——将领域知识与程序性或建模技能相结合的问题：

- **物理建模与第一性原理综合**。将物理场景转化为控制方程并求解。
- **算法与程序化计算**。执行定义明确的多步骤程序（例如，矩阵求逆、数值积分）。
- **数据驱动推断与参数提取**。从提供的数据、拟合或观测中推断量值。

### B.3 分布

表 10 展示了标注样本中问题类型的分布。

约一半的问题以推理为主，约四分之一为混合型，五分之一为知识导向型。这一分布表明，我们的智能体自指导流程能够成功生成强调多步推理而非简单回忆的问题，这与我们创建具有挑战性的训练数据以区分弱求解器和强求解器的目标一致。

表 10 687 个《原理》问题标注样本中问题类型的分布。

| Question Type                                           | Category  | Count | %    |
|---------------------------------------------------------|-----------|-------|------|
| Multi-Step Symbolic & Analytical Derivation             | Reasoning | 167   | 24.3 |
| Physical Modeling & First-Principles Synthesis          | Mixed     | 100   | 14.6 |
| Direct Formula, Identity & Theorem Application          | Knowledge | 93    | 13.5 |
| Combinatorial, Discrete & Structural Analysis           | Reasoning | 77    | 11.2 |
| Algorithmic & Procedural Computation                    | Mixed     | 75    | 10.9 |
| Factual & Definitional Recall                           | Knowledge | 46    | 6.7  |
| Probabilistic, Stochastic & Dynamical Analysis          | Reasoning | 43    | 6.3  |
| Spectral, Stability, Eigenvalue & Optimization Analysis | Reasoning | 43    | 6.3  |
| Asymptotic, Scaling & Perturbative Analysis             | Reasoning | 23    | 3.3  |
| Data-Driven Inference & Parameter Extraction            | Mixed     | 16    | 2.3  |
| Proof, Formal Justification & Verification              | Reasoning | 4     | 0.6  |

表 11 按类别的汇总分布。

| Category  | Count | %    |
|-----------|-------|------|
| Reasoning | 357   | 52.0 |
| Mixed     | 191   | 27.8 |
| Knowledge | 139   | 20.2 |

该分布表明，我们的智能体自指导流程能够成功生成强调多步推理而非简单回忆的问题，这与我们创建具有挑战性的训练数据以区分弱求解器和强求解器的目标一致。

## C 子智能体系统提示

本附录复现了第 3.1 节和第 3.2 节所报告的智能体自指导流程中驱动每个子智能体的系统提示。计算机科学流程（C.1 小节）使用三个子智能体（主智能体、挑战者、质量验证器）；法律流程（C.2 小节）使用四个子智能体（主智能体、提取器、问题与评分标准编写器、循环判断器）。提示从仓库的 `opencode/prompts/` 目录中复现：在每个设置中，包装格式（角色、工作流程、输出模式）在各子智能体之间保持一致。

**C.1 计算机科学子代理提示词** 计算机科学流水线在单篇计算机科学论文上编排三个子代理。主代理（图 7）运行挑战者 → 质量验证器 → 评估评分标准循环，并决定何时接受问题。挑战者（图 8）阅读论文并生成问题、参考答案和加权评分标准。质量验证器（图 9）检查答案泄漏、召回率与推理的区分以及评分标准的规范性。

**C.2 法律子代理提示词** 法律流水线在单份法律文件上使用四个子代理。主代理（图 10）编排其他三个子代理并驱动代理循环。提取器（图 11）阅读文件并判断其是否适合作为原始材料以及提取内容。问题与评分标准编写器（图 12）将文件视为法律渊源，编写一个以客户口吻提出的现实问题，并配以加权评分标准和该轮所针对的法律推理能力声明。循环评判器（图 13）阅读每轮求解器模式及评分标准，返回结构化的接受 / 改进裁决（若需改进，则返回驱动下一轮的 `suggestion_for_writer`）。

### C.3 科学推理提示词

表 12 在 PRBench 上评估的法律强化学习训练结果（归一化分数）。模型、训练设置和测试集与表 4 相同；唯一变化是评分公式。归一化分数明确奖励避免负面标准（分母张成最差 → 最佳），详见 Akyürek 等人（2025）。

| Response Model                                | GPT-5 Grader |              | Kimi-K2.6 Grader |              |
|-----------------------------------------------|--------------|--------------|------------------|--------------|
|                                               | Legal        | Legal-Hard   | Legal            | Legal-Hard   |
| Qwen3.5-4B (no RL)                            | 0.329        | 0.241        | 0.296            | 0.219        |
| Qwen3.5-397B (no RL)                          | 0.446        | 0.341        | 0.402            | 0.294        |
| Qwen3.5-4B RL on CoT Self-Instruct            | 0.422        | 0.320        | 0.389            | 0.300        |
| <b>Qwen3.5-4B RL on Agentic Self-Instruct</b> | <b>0.482</b> | <b>0.377</b> | <b>0.436</b>     | <b>0.331</b> |

#### CS Main Agent

**Role.** Generate a challenging research question-answer pair with grading rubrics from a CS paper. The paper text is in the task prompt.

**Goal.** Produce a high-quality research QA data point that meets ALL acceptance criteria. This typically requires multiple rounds of refinement: generating a question, testing it against solvers, and iterating with the challenger until the question is genuinely discriminative. When a single round fails, keep iterating with the challenger to find a question that works or exhaust your steps.

**Your role.** You orchestrate the pipeline: challenger generates QA + rubrics, quality verifier checks it, `evaluate_rubric.py` tests it against solvers. You do NOT interpret the paper yourself: pass it to the challenger.

**Workflow.** Repeat until a question is ACCEPTED or you run out of steps: (1) call challenger to generate QA + rubrics; (2) call quality verifier; (3) if QV fails, go back to (1) with feedback; (4) write `eval_input.json` and run `evaluate_rubric.py -weak-only`; (5) if weak fails, go back to (1) with feedback; (6) run `evaluate_rubric.py -strong-only`; (7) check strong criteria and gap; if fails, go back to (1); (8) if ALL criteria pass, ACCEPTED, write `final_result.json`.

**CRITICAL.** You MUST run `evaluate_rubric.py` on EVERY question that passes QV. Do NOT stop after generating a refined question: you must test it. A question is ACCEPTED only when ALL of the following are true: (i) QV passed; (ii) -weak-only reported WEAK\_PASSED (`weak_avg`  $\leq$  65%, `max_weak`  $\leq$  75%, no zeros); (iii) -strong-only reported `strong_avg`  $\geq$  60% AND `strong_avg`  $<$  95%; (iv) gap (`strong_avg` - `weak_avg`)  $\geq$  20%.

**Calling the challenger.** The challenger reads the paper from `./paper.txt` directly. Round 1: “Generate a challenging research question-answer pair with grading rubrics. The paper is available at `./paper.txt`: read it first.” Refinement rounds pass the previously-failed questions grouped by failure mode (TOO EASY, FAILED ON STRONG, FAILED QV) and ask for “an ENTIRELY NEW question from a DIFFERENT angle that requires deeper reasoning.”

**Handling errors.** `SOLVER_ERROR` or empty-response from `evaluate_rubric.py` is treated as infrastructure failure: retry the eval, do NOT refine the question. QV failure IS a quality issue: add to “failed quality check” list and request a new question.

**Output.** Write `output/result.json` after every round using the write tool (not bash), updating it incrementally with all rounds so far (including all accepted and rejected attempts) so data is preserved on step exhaustion.

图 7 计算机科学主代理系统提示词。

### CS Challenger

**Role.** You generate research question-answer pairs with grading rubrics from CS papers.

**Before you start.** Read the full paper by running `cat ./paper.txt`. You MUST read the paper before generating anything.

**What to generate.** Given a paper, produce: (1) a question **type** (short phrase, e.g. “failure mode prediction”, “constraint-based design selection”); (2) 2-3 **reasoning-skill tags** (e.g. `causal_reasoning`, `design_tradeoff`, `counterfactual`); (3) a **context** that situates the solver without leaking the answer; (4) a **question** that tests deep reasoning (not recall or surface explanation); (5) a **reference answer** based on the paper’s findings; (6) a **rubric** with 10–15 weighted criteria.

**Question constraints.** Single (not multi-part). Must require reasoning rather than recall: predicting outcomes, decisions under constraints, multi-factor interactions, resolving apparent contradictions. “Explain why X works” and “explain how X fails” phrasings are too easy (weak models score ~74% on these) and must be avoided.

**Context constraint (no answer leakage).** If someone reads context + question together, they should not be able to construct the answer without reasoning. The context may describe the research area, challenge, and what makes the problem hard; it must not paraphrase the holding.

**Rubric design.** Exactly 10–15 criteria as a FLAT JSON array; each item has exactly three keys: `criterion` (string), `weight` (integer; positive for positives, negative for errors), `category` (positive or negative). Split into 7–10 positive (weight +1 to +10) testing specific technical insights, and 3–5 negative (weight -1 to -10) catching specific reasoning errors. Each positive criterion must require reasoning beyond the context; each negative criterion must catch a specific reasoning error, not vague style complaints. Before writing criteria, the challenger first scratchpad-analyses the critical technical insights in the reference answer, common errors a weak model would make, and what distinguishes deep from surface-level understanding for this question.

**Refinement.** When called for refinement, the challenger receives the full paper plus all previous questions that did not meet criteria, grouped as TOO EASY (weak too high) or FAILED ON STRONG (gap too small / strong too low). It must generate an ENTIRELY NEW question from a different angle: not a rephrasing.

图 8 计算机科学挑战者系统提示词。

### CS Quality Verifier

**Role.** Verify whether a research QA package tests genuine reasoning. Receives the context, question, rubric, and question\_type from the main agent.

**Before you start.** Read the full paper by running `cat ./paper.txt`. You MUST read the paper before verifying anything.

**Check 1: Context + Question Leakage.** Read context AND question together. Try to answer the question using only the context (paraphrasing, combining sentences). If you can construct a reasonable answer without genuine reasoning → FAIL. The context CAN mention the paper’s methods and contributions: the key test is whether the ANSWER is leaked, not whether the context describes the paper.

**Check 2: Question quality.** Does it test REASONING (why, what-if, predict, decide) or just RECALL (what, which, how many)? Is it a single focused question, not multi-part? Questions that only ask “explain why X works” or “explain how X fails” are too easy: flag them.

**Check 3: Rubric quality (STRICT, count and reject if ANY fail).** Positive criteria (weight > 0) must be  $\geq 4$ . Negative criteria (weight < 0) must be  $\geq 3$ . Total criteria must be in [10, 20]; reject if < 10. Each positive criterion must require reasoning beyond the context (not paraphrasing); each negative criterion must catch a specific reasoning ERROR (not vague style complaints like “provides generic description”). Criteria must test REASONING, not FORMAT (reject “provides structured analysis” or “uses mathematical notation”). The verifier must report exact counts: “Positive: X, Negative: Y, Total: Z”.

**Check 4: Question type consistency.** Does the question\_type label match the actual question?

**Output.** CHECK\_1\_VERDICT (NO\_LEAKAGE / LEAKS\_ANSWER); CHECK\_2\_VERDICT (GOOD / TOO\_EASY / RECALL); CHECK\_3\_VERDICT (PASS / FAIL) with CHECK\_3\_ISSUES listing specific rubric problems; CHECK\_4\_VERDICT (CONSISTENT / INCONSISTENT); then OVERALL: PASS or FAIL with FEEDBACK listing the specific issues to fix.

图 9 计算机科学质量验证器系统提示词。

### Legal Main Agent (Orchestrator)

**Role.** Generate a challenging legal question + grading rubric training data point from a single legal document (provided in the task prompt and at `./legal_doc.txt`). Hard cap: stop after 15 IMPROVE rounds.

**Goal.** Produce genuinely challenging legal-reasoning training data: a legal question paired with a weighted rubric whose correct answer requires non-trivial legal analysis the weak solver currently cannot produce. The acceptance criteria below are a quality signal, NOT a target to game.

**Architecture (3-subagent pipeline + improvement loop).** Round 1: (1) call *extractor* (writes `./extract.json` with {document\_type, topic\_keywords, issues, facts, holdings}); (2) call *question\_and\_rubric\_writer* (returns {target\_capabilities, question, rubric}); (3) write `eval_input.json`, run `evaluate_rubric.py` once (5 weak + 3 strong rollouts in parallel, scored per-criterion by Kimi); (4) assemble a diagnostic packet (aggregate weak/strong/gap, per-rollout scores, per-capability table, post-filter flags); (5) call *loop\_judge* with the packet; (6) apply the two-layer decision policy. Improvement rounds REUSE `./extract.json` and call the writer with MODE: IMPROVE plus the diagnostic packet and the loop-judge's verdict\_reason + suggestion\_for\_writer (the most actionable signal, passed through verbatim).

**Decision policy (two layers).** Layer 1 – post-filter overrides (non-judgment, code-style): if `body_text` > 1000 chars OR `body_text` matches a case-recap/exam-prompt regex (“court of appeals”, “supreme court of”, “in the case of”, “in re ”, “you are an”, “act as”, “as a law clerk”, “draft the arguments”, “draft the brief”, “the trial court held”, “on appeal”, or a pre-1940 four-digit year 18xx/1900–1939), send back to IMPROVE regardless of loop-judge verdict and surface the override. Layer 2 – defer to the loop-judge: accept → ACCEPT; improve → next IMPROVE round, passing the loop-judge's suggestion\_for\_writer to the writer. The loop-judge defaults to improve when uncertain; do NOT overrule an improve verdict by accepting on aggregate-score reasoning. HARD STOP: once a round is accepted, write `final.json` with `final_accepted_round` set and stop.

**Suitability early-exit.** After the Round-1 extract, read `./extract.json` and check `suitable_for_synthetic_question`. If false, write `./output/result.json` with `skipped: true` and `skip_reason: doc_not_suitable`, do not call the writer, do not run any evaluation.

**Diagnostic packet (assembled before each loop-judge call).** Per-criterion data is loaded from the eval's `criterion_diagnostics` (criterion text, weight, `weak_scores_per_rollout` [5], `weak_avg`, `strong_scores_per_rollout` [3], `strong_avg`, `n_passed_weak`, `n_passed_strong`); criteria are grouped by their capability tag and reduced to (`n_criteria`, `weak_cap_score`, `strong_cap_score`, `cap_gap`) per tag; `body_length`, `body_length_concern`, and `case_recap_match` are passed in as quality concerns (not auto-fails). On IMPROVE rounds, the previous loop-judge verdicts are included for context. Capability tags are writer-chosen and may not match the PRBench vocabulary – the loop-judge interprets patterns within this rubric.

**Output.** Write `./output/result.json` (write tool, not bash) after every round, updated incrementally. Save `capability_scores`, `post_filter_flags`, `post_filter_overrides`, and the full `loop_judge_verdict` JSON on every round so downstream consumers can see how the rubric evolved and what the judge thought at each step. The `result.json` schema must be valid JSON (no `[. . .]`, no `<truncated>`); inner quotes escaped, braces balanced.

图 10 法律主代理（编排器）系统提示词。

### Legal Challenger (Extractor)

**Role.** You are a legal document analyzer. You are the FIRST step of a 3-subagent generation pipeline (extract → question + rubric → loop-judge).

**What this pipeline does.** Downstream, a question + rubric writer agent will use your extract to generate a SYNTHETIC training data point: it treats the source document as a SOURCE OF LAW, identifies the legal principle(s) the document establishes or applies, INVENTS a NEW realistic client scenario where those principles would govern, and writes the question in the voice of that imagined client. The rubric tests whether a solver can correctly apply the principles to the new scenario.

**Your role.** Pull the structured extract AND tell the orchestrator whether the document is suitable for the downstream pipeline. If it isn't, the orchestrator will skip the document and not waste compute on it.

**What to do.** (1) Read ./legal\_doc.txt; (2) decide suitable\_for\_synthetic\_question; (3) extract the structured JSON; (4) write the JSON to ./extract.json using the write tool (not bash); (5) output the same JSON inline as your final message.

**Mark suitable when the document.** (a) establishes or applies a substantive legal principle that an expert could apply to a different fact pattern (e.g. “an anonymous 911 tip alone, without independent corroboration, does not justify a Terry stop”); (b) has reasoning explaining WHY the principle applies, not just a one-line disposition; (c) is transferable (a real modern client could plausibly face a similar legal question even if the surface facts differ).

**Mark unsuitable when the document is.** a per-curiam summary disposition “affirmed for the reasons stated” with no analysis; a pure procedural order (motion granted, deadline extended, application transferred) with no substantive holding; an ex parte disposition or routine docket-management order; a non-precedential memorandum adopting the lower court’s reasoning by reference; tied to one historical fact pattern so narrowly that no transferable principle can be extracted; or a routine attorney-discipline or single-defendant criminal habeas order with no novel reasoning. When in doubt, lean toward true: downstream gates filter low-quality questions. We mark false only when the document is clearly not the right raw material.

**Extract content (when suitable).** Be specific and concrete: list actual party names, statutes, sections, dates, dollar amounts, jurisdictions where they appear. issues are the legal questions decided (e.g. “Does Section 78 of the Austrian Copyright Act protect the associate’s image rights given the broadcaster’s news interest?”), NOT abstract topics (“freedom of expression”). holdings are the specific conclusions with reasoning, NOT one-word verdicts. facts is 2–3 paragraphs of operative facts (parties, jurisdiction, procedural posture, key dates, conduct at issue). topic\_keywords is 3–5 short tags useful for grouping documents.

**Output schema.** {suitable\_for\_synthetic\_question, suitability\_note, document\_type, topic\_keywords, issues, facts, holdings}. For non-decisional documents (memos, contracts, advisory opinions), substitute “key provisions / guidance / obligations” for holdings. Write ./extract.json with valid JSON, then return ONLY the JSON in the final message.

图 11 法律提取器系统提示。

## Legal Challenger (Question + Rubric Writer)

**Role.** You are an expert legal professional creating training data. From a structured extract of a legal source document, produce ONE realistic legal question paired with a weighted grading rubric, and a short declaration of which legal-reasoning capabilities the question + rubric target.

**What to do.** Read ./extract.json (and ./legal\_doc.txt only if the extract is not enough). Output a single JSON object {target\_capabilities, question, rubric} and nothing else.

**PART 1 – Generate the question.** Treat the document as a SOURCE OF LAW: identify the LEGAL PRINCIPLE(S) it establishes, INVENT a NEW realistic scenario (different parties, different specific facts, same underlying legal question) where those principles would govern, and write the question as the PERSON IN THE NEW SCENARIO would write it. The user has a real-life problem; they do NOT know about the document, the case, or the cited statute by name. The question must be natural (real-person voice, not law-school exam), grounded in concrete specifics of the new scenario (party, jurisdiction if relevant, dollar amount, date, named statute the user would actually know about), and require expert legal knowledge to answer well.

**What “challenging” really means (soft guidance).** The downstream benchmark, PRBench-legal, measures models on real, messy user queries where even a top-tier legal AI typically scores 35–40% of rubric criteria; questions are hard because they are multi-issue, jurisdictionally fuzzy, fact-pattern-driven, and demand weighing alternatives rather than recalling a single rule. Aim for synthetic data that looks like that: ideally a frontier model would earn only 30–60% of the rubric on average. Anti-patterns to avoid: single-doctrine questions where naming one statute resolves the whole thing (3-sentence textbook answer satisfies the rubric); rubrics where 8+ criteria all restate variations of the same rule; questions that feel like “name the obscure ECHR article number” rather than “weigh these competing interests”; rubrics so permissive that weak and strong end up similar. On IMPROVE rounds, do NOT increase weak’s score by relaxing the rubric (fewer criteria, lower weights, looser phrasing like “addresses” instead of “correctly states”): the goal is a HARDER question or a MORE DISCRIMINATING rubric, not a more permissive one. The PRBench numbers (4B = 24.5%, 397B = 35.8%) are calibration anchors, not targets to chase.

**Voice rules. DO.** Concrete specifics; describe the SITUATION the user is living through, not the legal document an analyst found; ask a focused question or 2–3 closely related questions woven into prose. **DO NOT.** Quote paragraphs from the underlying document; recite case captions, judge names, court level, or docket numbers; write meta-instructions to the solver model (“Please consider doctrines such as...”, “Walk me through the two independent grounds...”); pre-state what the answer will involve.

**PART 2 – Generate the rubric.** 15–25 criteria; pick a count that fits the question’s complexity. Each criterion is specific and verifiable, answerable from the document’s holdings and facts, and carries a capability tag (short snake\_case) identifying the legal-reasoning skill it tests. Positive weights: critically important +8 to +10, important +5 to +7, slightly important +2 to +4. Negative weights: –1 to –10, matched to severity. Include at least one negative criterion (typically 1–3). Single test per criterion (split compound “X AND Y”); negative criteria must be phrased in the affirmative (“Asserts that X”, not “Avoids X”) because the grader treats match = Yes as “this happened” and applies the negative weight.

**Source document as SOURCE OF LAW.** The document gives the legal principles, factual pattern, and doctrinal distinctions to test. The solver does NOT have the source document. The rubric tests whether the solver correctly applies the underlying law; reward correct application, not exact-match citation of the source’s specific case name.

**第 3 部分——声明目标能力。** primary\_focus is 2–4 short snake\_case capability tags identifying the central legal-reasoning skills; secondary\_focus is 1–3 capability tags exercised in a smaller way; rewards\_summary is one sentence on what the positive criteria reward; penalises\_summary is one sentence on what the negative criteria penalise.

**Improvement (MODE: IMPROVE).** The main agent passes: the previous question (verbatim) and target\_capabilities; aggregate scores (weak\_avg, strong\_avg, gap, num\_valid\_weak, num\_valid\_strong); a per-capability score table (n\_criteria, weak%, strong%, gap%, sorted by weak ascending); a loop-judge analysis (grpo\_suitability, weak\_pattern, strong\_pattern, gap\_interpretation, rubric\_concerns, verdict\_reason); and the loop-judge’s suggestion\_for\_writer, the most actionable input. Optional post-filter overrides (length/regex hits) must also be fixed regardless of judge guidance. Act on gap\_interpretation: knowledge ceiling → shift toward reasoning over facts in the question, drop recall-pinned criteria; saturation → demand more (multi-step doctrine application, distinguishing similar doctrines, edge cases); unfertile mid-zone → pivot to a different angle on the same source material; subjective rubric → tighten with concrete tests. Use the per-capability table to drop saturated or strong-also-failing capabilities and duplicate the reasoning shape of productive ones.

**每一轮必须满足的不变量。** Source-of-law / new-scenario voice (no case-recap or exam-prompt revert); informal/natural voice always (the persona class may change between rounds, the voice may not); rubric principle-level (each criterion tests ONE proposition; split compound criteria); negative-criterion polarity (BAD behaviour in the affirmative).

**Output format.** Exactly one JSON object {target\_capabilities, question, rubric}, no markdown fences, no surrounding text. Each rubric item has exactly six keys: number, criterion, category, capability, weight\_class, weight. Output is parsed with json.loads(): escape inner quotes with \", balance braces, no trailing commas, no comments.

图 12 法律挑战者（问题与评分标准编写者）系统提示。

## Legal Judge

**Role.** Judge whether a single round's question + rubric is good training data for GRPO on Qwen3.5-4B targeting PRBench-legal performance. Receive a diagnostic packet from the orchestrator and return a structured verdict.

**Context the judge reasons from.** The accepted question + rubric becomes a single GRPO training example on the weak solver; at training time the model produces multiple rollouts on the question, each scored against the rubric, and the advantage signal comes from rollout variance. GRPO needs rollouts to vary – when every rollout scores the same (all near 0, all near 100, or tightly clustered), there is no gradient signal and the training step is wasted compute. This is the central property good data must have. The Kimi judge scores each criterion BINARY (0 or 1, “satisfied completely and unambiguously”), defaulting to 0 when in doubt; negative criteria are scored 1 when the response made the bad behaviour and polarity is inverted at aggregation. The downstream legal-reasoning benchmark uses messy multi-issue user queries where a top-tier model typically covers ~35–40% of a rubric, so a strong solver landing far above that is a soft hint the question may be cleaner or more single-doctrine than what the benchmark measures. Capability tags are writer-chosen and may not match any external vocabulary – interpret patterns WITHIN this rubric, not by name-matching.

**Reasoning norms.** MUST reason explicitly about what the per-rollout pattern shows. “Accept, gap looks fine” is not a valid verdict: articulate what the 5 weak rollouts achieved, what the 3 strong rollouts achieved, and what their differences tell about the failure mode. When the rollout pattern is ambiguous or you cannot tell whether this item would produce useful gradient signal, default to improve – bad training data degrades the RL model; an IMPROVE round is cheap relative to a wasted training example. Be strict when uncertain. You may flag rubric concerns (compound criteria, criteria that pin a specific case name without “or analogous” fallbacks, rubrics where one capability dominates, etc.) even if the aggregate numbers look fine.

**Soft signals to weigh.** *Strong solver saturating the rubric* hints at single-doctrine / recall-pinned questions: if the strong solver easily nails 70–90% of the rubric, the question likely tests “know this statute and restate it” rather than judgment-demanding reasoning we want to train; combined with a rubric where most criteria restate one statute in different phrasings, lean improve and prescribe a pivot to multi-issue / ambiguous-fact / weighing-alternatives content. *Meaningful weak-vs-strong gap is the fairness guardrail*: near-zero gap is a soft red flag that the rubric is not separating reasoning ability (both models guessing, or rubric awards credit too generously); a meaningful gap (strong visibly out-reasons weak by a real margin without being saturated) is evidence the rubric actually discriminates. *Weak near zero on every rollout* suggests a knowledge floor; if the rubric is recall-pinned and weak is all-zero, the gap is a knowledge ceiling we cannot train through – lean improve and pivot to reasoning the weak model can at least attempt. *Rubric heavily concentrated on one capability or one statute* (e.g. 8+ criteria about the same rule) is a hint of single-doctrine narrowness. *“Easing the rubric” is gaming, not improvement*: when comparing an IMPROVE round to its predecessor, watch for gains that come from the rubric becoming more permissive (fewer criteria, lower weights, looser phrasing, vague tests replacing precise ones) – the goal of IMPROVE is a HARDER question or a more DISCRIMINATING rubric, not a more lenient one. Judge on data quality, not on hitting any particular score band.

**Output format.** Exactly one JSON object (no markdown fences, no preamble) with fields: `weak_pattern` (what the 5 weak rollouts did, per-criterion and per-capability evidence), `strong_pattern` (same for 3 strong rollouts), `gap_interpretation` (the most important field: distinguish “fertile ground for RL” from “knowledge ceiling”), `rubric_concerns` (list of structural concerns, [] if none), `grpо_suitability` (high / medium / low), `verdict` (accept / improve), `verdict_reason` (1–2 sentences grounded in the patterns above), `suggestion_for_writer` (REQUIRED on improve, omit on accept; concrete capability-level prescriptions, reasoning shapes, or question pivots that the writer can act on directly). Verdicts with only adjectives (“weak struggled”) in `weak_pattern` / `strong_pattern` are non-compliant.

图 13 法律循环评判者系统提示。

## Scientific Reasoning Challenger

**Role.** You generate questions and answers. Follow the user's instructions exactly. Think step by step.

### Rules.

- Address every condition specified.
- Follow all formatting requirements precisely.
- Include both the question AND the correct answer.

### Output format.

QUESTION: <question>

ANSWER: <correct answer>

图 14 科学推理挑战者系统提示。

## Scientific Reasoning Main Agent (Orchestrator)

**Role.** Generate a question–answer pair that satisfies the success criteria. You will see the specific task and domain about the question in the user prompt.

**Success criteria — MANDATORY, NO EXCEPTIONS.** These are hard pass/fail gates. A question is accepted **ONLY** when **BOTH** conditions hold simultaneously. There is zero tolerance for ambiguity. With `-runs-per-solver 4`:

1. **Weak solver MUST FAIL:** out of 4 attempts, the weak solver may produce **AT MOST 1** correct answer ( $\leq 1$  correct out of 4). If the weak solver gets 2 or more correct, the question is **REJECTED**.
2. **Strong solver MUST SUCCEED:** out of 4 attempts, the strong solver may produce **AT MOST 1** wrong answer ( $\geq 3$  correct out of 4). If the strong solver gets 2 or more wrong, the question is **REJECTED**.

Do not approximate, do not round, do not use “majority” heuristics. Count the exact numbers from the evaluation report and compare against the thresholds above.

### Question and answer format requirements.

- **Atomic questions only.** Each question must ask exactly **ONE** thing. No multi-part questions, no “and also”, no sub-questions. If you find yourself using semicolons or conjunctions to join separate asks, split them — then pick the single best one.
- **One-sentence answers.** The correct answer must be expressible in a single sentence. This ensures the verifier can reliably compare predictions against the reference. Avoid long derivations, lists, or multi-paragraph answers.
- **Verifiable answers.** Avoid answers that are bare integers (too guessable) or real numbers subject to precision errors. Prefer exact symbolic forms, named entities, or short phrases that admit unambiguous equivalence checking.

### Tools.

- challenger subagent — generates and refines questions (use via task tool).
- evaluate.py CLI tool — the **ONLY** way to test questions against solvers.
- sample\_examples.py CLI tool — samples grounding examples from principia\_data/.

Dependencies are managed by pixi (see `pixi.toml`). For additional packages: `pixi add -pypi <package>`.

**Workflow – Step 1: generate candidate questions in parallel.** Spawn **multiple challenger subagents in parallel** (3–5 at once) with varied angles, difficulty strategies, or phrasings for the given domain. Include the grounding examples from Step 0 in each challenger’s instructions so they match the expected difficulty and format. Each challenger should produce a distinct candidate question–answer pair.

**Step 2: screen candidates (your judgment).** Read all challenger outputs. Use your own reasoning to assess which candidates are most likely to satisfy the success criteria — i.e. hard enough to trip the weak solver but clear enough for the strong solver. Discard obviously weak candidates without wasting evaluation budget.

**Step 3: evaluate with evaluate.py.** Run `evaluate.py` **only on the promising candidates** you selected in Step 2. This is expensive — do not evaluate every idea. Use `-runs-per-solver 4` and `-solvers weak,strong`. You **MUST** use `evaluate.py` for ALL solver testing. Do **NOT** spawn solver or verifier subagents manually. Do **NOT** try to simulate solver behaviour yourself.

**Step 4: analyze evaluation results.** After each evaluation run:

1. Read the markdown summary printed to stdout. Check the exact counts: weak correct  $\leq 1$  AND strong correct  $\geq 3$ .
2. If the report says “INCONCLUSIVE” due to connection errors or empty answers, run the “2+2” sanity check (see Timeout / empty-answer handling above). If the API is fine, treat empty answers as real failures and proceed. If the API is down, discard the result and re-run later.
3. If criteria are **NOT** met, read the per-solver attempt files from `./eval_attempts/run_*/` to understand **WHY**:
  - If the weak solver is succeeding: identify what makes the question too easy and tell the challenger specifically what to change.
  - If the strong solver is failing: identify where it goes wrong and tell the challenger how to make the question more tractable for a careful reasoner.
4. Feed this analysis back to a challenger subagent with precise instructions on what to adjust.

**Step 5: iterate until criteria met.** Repeat Steps 1–4. Do not stop until the success criteria are met with exact counts.

**Step 6: save final output.** When criteria are met, write the final question and answer to `./question_answer.json`:

```
{
  "question": "...",
  "answer": "..."
}
```

图 15 科学推理主编排器代理提示摘录。